

# The Complexity of Boolean Conjunctive Queries with Intersection Joins

Mahmoud Abo Khamis  
RelationalAI  
Berkeley, United States

Antonia Kormpa  
University of Oxford  
Oxford, United Kingdom

George Chichirim  
University of Oxford  
Oxford, United Kingdom

Dan Olteanu  
University of Zurich  
Zurich, Switzerland

## ABSTRACT

Intersection joins over interval data are relevant in spatial and temporal data settings. A set of intervals join if their intersection is non-empty. In case of point intervals, the intersection join becomes the standard equality join.

We establish the complexity of Boolean conjunctive queries with intersection joins by a many-one equivalence to disjunctions of Boolean conjunctive queries with equality joins. The complexity of any query with intersection joins is that of the hardest query with equality joins in the disjunction exhibited by our equivalence. This is captured by a new width measure called the *ij-width*.

We also introduce a new syntactic notion of acyclicity called *iota-acyclicity* to characterise the class of Boolean queries with intersection joins that admit linear time computation modulo a poly-logarithmic factor in the data size. *Iota-acyclicity* is for intersection joins what *alpha-acyclicity* is for equality joins. It strictly sits between *gamma-acyclicity* and *Berge-acyclicity*. The intersection join queries that are not *iota-acyclic* are at least as hard as the Boolean triangle query with equality joins, which is widely considered not computable in linear time.

## CCS CONCEPTS

• **Theory of computation** → Database query processing and optimization (theory).

## KEYWORDS

interval joins; spatiotemporal data; segment tree; *iota* acyclicity

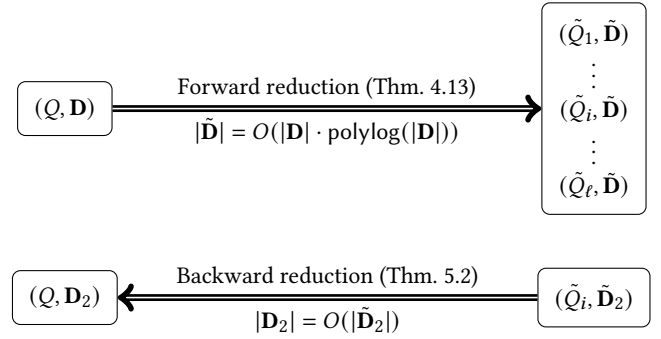
## ACM Reference Format:

Mahmoud Abo Khamis, George Chichirim, Antonia Kormpa, and Dan Olteanu. 2022. The Complexity of Boolean Conjunctive Queries with Intersection Joins. In *Proceedings of the 41st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS '22)*, June 12–17, 2022, Philadelphia, PA, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3517804.3524156>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

PODS '22, June 12–17, 2022, Philadelphia, PA, USA

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.  
ACM ISBN 978-1-4503-9260-0/22/06...\$15.00  
<https://doi.org/10.1145/3517804.3524156>



**Figure 1: Forward and backward reductions from Sections 4 and 5 respectively.**  $Q$  is a query with intersection joins,  $\tilde{Q}_1, \dots, \tilde{Q}_l$  are queries with equality joins,  $\mathbf{D}$  and  $\mathbf{D}_2$  are databases of intervals,  $\tilde{\mathbf{D}}$  and  $\tilde{\mathbf{D}}_2$  are databases of numbers.

## 1 INTRODUCTION

Interval data is common in spatial and temporal databases. One important type of joins on intervals is the intersection join: A set of intervals join if their intersection is non-empty. In the case of point intervals, the intersection join becomes the classical equality join.

This paper establishes the complexity of Boolean conjunctive queries with Intersection Joins (denoted by IJ). Whereas the complexity of Boolean conjunctive queries with Equality Joins (denoted by EJ) has been extensively investigated in the literature by, e.g., Marx [21] and Abo Khamis et al. [4], the complexity of the more general IJ queries remained open for decades.

The key tool aiding our investigation is a many-one equivalence of any IJ query to a disjunction of EJ queries. It uses a forward reduction (IJ-to-EJ) and a backward reduction (EJ-to-IJ), cf. Figure 1.

The *forward* reduction takes an IJ query  $Q$  and a database  $\mathbf{D}$  of intervals. It reduces  $Q$  to a disjunction of EJ queries, all over the same database  $\tilde{\mathbf{D}}$  of numbers represented as bitstrings. The number and size of the EJ queries only depend on the structure of  $Q$ , and the size of  $\tilde{\mathbf{D}}$  is within a poly-logarithmic factor from the size of  $\mathbf{D}$ . This forward reduction enables us to use any algorithms and associated runtime *upper bounds* for the EJ queries as upper bounds for the IJ query  $Q$  as well. Specifically,  $Q$ 's runtime is upper bounded by the maximum runtime upper bound among the generated EJ queries.

The *backward* reduction takes an EJ query  $\tilde{Q}_i$ , whose structure matches that of one of the queries obtained by the forward reduction of an IJ query  $Q$  without self-joins, and an arbitrary database  $\tilde{\mathbf{D}}_2$

of numbers chosen independently from  $\tilde{\mathbf{D}}$  and  $\mathbf{D}$  in the forward reduction. It then reduces  $\tilde{Q}_i$  to an IJ query  $Q$  whose structure matches that of the original IJ query  $Q$ , and reduces  $\tilde{\mathbf{D}}_2$  to some database  $\mathbf{D}_2$  of intervals and size  $O(|\tilde{\mathbf{D}}_2|)$ . The backward reduction shows that we can use any *lower bounds* (i.e. hardness results) on any one of the EJ queries  $\tilde{Q}_i$  constructed by the forward reduction as lower bounds on the IJ query  $Q$ . Together with the upper bounds, this implies that the IJ query  $Q$  is *precisely* at the same hardness level as the hardest EJ query  $\tilde{Q}_i$ . Our forward reduction thus produces an optimal solution to  $Q$  given optimal solutions to the queries  $\tilde{Q}_i$ .

The quest for the optimality of EJ computation has a long history. The submodular width has been recently established as an optimality yardstick [4, 21]. Let  $BCQ(C)$  be the decision problem: Given an EJ query  $Q \in C$  and a database  $\mathbf{D}$ , check whether  $Q(\mathbf{D})$  is true. This problem is fixed-parameter tractable (FPT; with parameter the query size  $|Q|$ ) if there is an algorithm solving every  $C$ -instance in time  $f(|Q|) \cdot |\mathbf{D}|^d$  for some fixed constant  $d$  and any computable function  $f$ . The problem  $BCQ(C)$  is FPT if and only if every  $Q \in C$  has bounded submodular width [21].

A natural question is then what would be an optimality yardstick for IJ computation. We settle this question with a new width notion called the ij-width. This is the maximum submodular width [21] of the EJ queries obtained by our forward reduction. Any IJ query  $Q$  can then be computed in time  $O(N^w \text{polylog } N)$ , where  $N$  is the size of the input database and  $w$  is the ij-width of  $Q$ . Each EJ query created by our forward reduction is computed over databases of size  $O(N \text{polylog } N)$  and in time  $O(N^w \text{polylog } N)$ .

The relationship between the complexities of IJ and EJ is non-trivial. In general, IJ queries are, as expected, more expensive than their EJ counterparts. There are simple  $\alpha$ -acyclic IJ queries whose complexity is on par with that of cyclic EJ queries. Consider the EJ counterparts of IJ queries, where we replace the intersection joins by equality joins. The triangle IJ query has ij-width  $3/2$  (Section 1.1), which matches the submodular width of the EJ triangle query. Yet for the IJ Loomis-Whitney query with four variables, the ij-width is  $5/3$  while the EJ counterpart has submodular width  $4/3$ .

This raises the question of what is the natural counterpart of  $\alpha$ -acyclic EJ queries, which are known to be *the* linear-time computable EJ queries [29]. We introduce an acyclicity notion called *iota* ( $\iota$ ) that syntactically characterises the class of IJ queries computable in linear time (modulo a polylog factor): An IJ query is  $\iota$ -acyclic if and only if the incidence graph of its hypergraph does not have Berge cycles of length strictly greater than two. Our forward reduction maps  $\iota$ -acyclic IJ queries to an equivalent disjunction of  $\alpha$ -acyclic EJ queries, so by definition the ij-width of any  $\iota$ -acyclic IJ query is one.  $\iota$ -acyclicity implies  $\gamma$ -acyclicity and is implied by Berge-acyclicity. The IJ queries that are not  $\iota$ -acyclic are at least as hard as the EJ triangle query, which is widely considered not computable in linear time (based on the 3SUM conjecture) [25].  $\iota$ -acyclicity is thus for IJ queries what  $\alpha$ -acyclicity is for EJ queries.

An intersection join can also be expressed as a disjunction of inequality joins, which can be evaluated using FAQ-AI [2]. However, FAQ-AI has a higher complexity than our approach: The exponents in the time complexity of FAQ-AI for the IJ triangle, Loomis-Whitney four, and the clique-four queries are: 2, 2, and respectively 3; whereas the ij-widths are:  $3/2$ ,  $5/3$ , and respectively 2.

## 1.1 Example: The Triangle Query

We introduce our approach using the Boolean triangle query, where each join is an intersection join:

$$Q_{\Delta} = R([A], [B]) \wedge S([B], [C]) \wedge T([A], [C])$$

Brackets denote interval variables ranging over intervals with real-valued endpoints. The two occurrences of  $[A]$  in the query denote an intersection join on  $A$ . An equality join on  $A$  is expressed using the variable  $A$  without brackets.

A common approach first computes the join of two of the three relations and then joins with the third relation. The first join can take  $O(N^2)$ , where  $N$  is the size of the relations. An equivalent encoding of  $Q_{\Delta}$  using inequality joins can be computed in time  $O(N^2 \log^3 N)$  using FAQ-AI [2] (Appendix E). Our approach takes time  $O(N^{3/2} \log^3 N)$ , which matches the complexity of the EJ triangle query (modulo polylog factor).

Our approach is based on a decomposition of the tensor representing an intersection join. In our example, we decompose the three joins as follows. We construct three segment trees: one for the intervals from  $R$  and  $T$  for the interval variable  $[A]$ , another for the intervals from  $R$  and  $S$  for the interval variable  $[B]$ , and the third one for the intervals from  $S$  and  $T$  for the interval variable  $[C]$ . The segment tree for  $O(N)$  intervals can be constructed in  $O(N \log N)$  time and has depth  $O(\log N)$ . Its nodes represent intervals called segments. A segment includes its descendant segments and is partitioned by its child segments. A property of a segment tree is that each input interval can be expressed as the disjoint union of at most  $O(\log N)$  segments. The problem of checking whether two input intervals intersect now becomes the problem of finding two segments, one per interval, that lie along the same root-to-leaf path in the segment tree. There are three ways this can happen: The two segments are the same, one segment is an ancestor of the other, or the other way around; this corresponds to the possible permutations of the two segments along a path, with one permutation also including the case where the segments are the same. A further property is that each permutation can be expressed using equality joins, as explained next. We encode each node in the segment tree as a bitstring: the empty string represents the root, the strings "0" and "1" represent the left and right child respectively, the strings "00" and "01" represent the left and right child of the "0" respectively, and so on. Given two nodes  $n_1$  and  $n_2$ , where  $n_1$  is an ancestor of  $n_2$ , the bitstring for  $n_1$  is then a prefix of that for  $n_2$ . We can capture this relationship in a query with equality joins: We use one variable  $A_1$  to stand for the bitstring for  $n_1$ , which is also a prefix of the bitstring for  $n_2$ , and  $A_2$  to stand for the remaining bitstring for  $n_2$ .

There are eight possible configurations for the ancestor-descendant relationship between the segment tree nodes for the two  $A$ -intervals in  $R$  and  $S$  and similarly for  $B$  and  $C$ . Each such case can be expressed using an EJ query. Our query  $Q_{\Delta}$  is then equivalent to the disjunction  $\tilde{Q}_{\Delta} = \bigvee_{i \in [8]} \tilde{Q}_i$  of the following EJ queries:

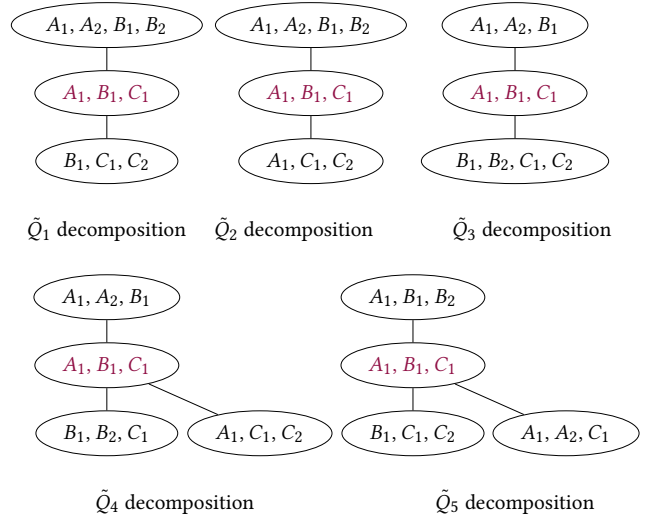
$$\begin{aligned} \tilde{Q}_1 &= R_{2,2}(A_1, A_2, B_1, B_2) \wedge S_{1,2}(B_1, C_1, C_2) \wedge T_{1,1}(A_1, C_1) \\ \tilde{Q}_2 &= R_{2,2}(A_1, A_2, B_1, B_2) \wedge S_{1,1}(B_1, C_1) \wedge T_{1,2}(A_1, C_1, C_2) \\ \tilde{Q}_3 &= R_{2,1}(A_1, A_2, B_1) \wedge S_{2,2}(B_1, B_2, C_1, C_2) \wedge T_{1,1}(A_1, C_1) \end{aligned}$$

$$\begin{aligned}
\tilde{Q}_4 &= R_{2;1}(A_1, A_2, B_1) \wedge S_{2;1}(B_1, B_2, C_1) \wedge T_{1;2}(A_1, C_1, C_2) \\
\tilde{Q}_5 &= R_{1;2}(A_1, B_1, B_2) \wedge S_{1;2}(B_1, C_1, C_2) \wedge T_{2;1}(A_1, A_2, C_1) \\
\tilde{Q}_6 &= R_{1;2}(A_1, B_1, B_2) \wedge S_{1;1}(B_1, C_1) \wedge T_{2;2}(A_1, A_2, C_1, C_2) \\
\tilde{Q}_7 &= R_{1;1}(A_1, B_1) \wedge S_{2;2}(B_1, B_2, C_1, C_2) \wedge T_{2;1}(A_1, A_2, C_1) \\
\tilde{Q}_8 &= R_{1;1}(A_1, B_1) \wedge S_{2;1}(B_1, B_2, C_1) \wedge T_{2;2}(A_1, A_2, C_1, C_2)
\end{aligned}$$

The query is defined over new relations  $R_{\mathcal{A};\mathcal{B}}$ ,  $S_{\mathcal{B};\mathcal{C}}$ , and  $T_{\mathcal{A};\mathcal{C}}$  that are transformations of the original relations  $R$ ,  $S$ , and  $T$  to hold the bitstrings in place of the original intervals. The subscript  $\mathcal{A} = i$  stands for the indices  $1, \dots, i$  of the variables  $A_1, \dots, A_i$  ranging over bitstrings; similarly for  $\mathcal{B}$  and  $\mathcal{C}$ . To avoid clutter, in the remainder of this paper we will denote all such constructed new relations  $R_{\mathcal{A};\mathcal{B}}$  by  $\tilde{R}$  and use their schema to identify them uniquely. For input relations of size  $N$ , the new relations have size  $O(N \log^2 N)$ , with one logarithmic factor per join interval variable. We next explain the purpose of these bitstring relations. A tuple  $(a_1, a_2, b_1, b_2)$  is in  $R_{2;2}$  if and only if there is a pair of intervals  $(a_R, b_R)$  in  $R$  and the concatenations  $a_1 \circ a_2$  and  $b_1 \circ b_2$  reconstruct bitstrings of nodes covered by intervals  $a_R$  and respectively  $b_R$ . Similarly, a tuple  $(b_1, c_1, c_2)$  in  $S_{1;2}$  reconstructs the bitstrings  $b_1$  and  $c_1 \circ c_2$  of nodes covered by intervals  $b_S$  and  $c_S$ . A tuple  $(a_1, c_1)$  in  $T_{1;1}$  specifies the bitstrings  $a_1$  and  $c_1$  of nodes covered by  $a_T$  and  $c_T$ . Therefore,  $\tilde{Q}_1$  holds in case: (1) there exists a node covered by an interval  $a_R$  in  $R$  and a node covered by an interval  $a_T$  in  $T$  such that the bitstring of the latter is a prefix of the former; (2) there exists a node covered by an interval  $b_R$  in  $R$  and a node covered by an interval  $b_S$  in  $S$  such that the bitstring of the latter is a prefix of the former; and (3) there exists a node covered by an interval  $c_S$  in  $S$  and a node covered by an interval  $c_T$  in  $T$  such that the bitstring of the latter is a prefix of the former. Equivalently,  $a_R$  intersects with  $a_T$ ,  $b_R$  intersects with  $b_S$ , and  $c_S$  intersects with  $c_T$ , i.e.,  $R(a_R, b_R) \wedge S(b_S, c_S) \wedge T(a_T, c_T)$  holds.

The hypergraph of each of the eight EJ queries admits a hypertree decomposition in the form of a star with the central bag  $\{A_1, B_1, C_1\}$ , cf. Figure 2. In each of these decompositions, the materialisation of this bag requires solving the triangle join  $R'(A_1, B_1) \wedge S'(B_1, C_1) \wedge T'(A_1, C_1)$ , where  $R'$  is a projection of  $R_{\mathcal{A};\mathcal{B}}$  to  $A_1, B_1$  and similarly for  $S'$  and  $T'$ . The new relations and their projections have size  $O(N \log^2 N)$ . The materialisation of the join takes time  $O((N \log^2 N)^{3/2}) = O(N^{3/2} \log^3 N)$  using existing worst-case optimal join algorithms [22]. Checking whether any of the eight EJ queries is true takes time linear in the maximum size of the bags of its decomposition. This gives an overall computation time  $O(N^{3/2} \log^3 N)$  for  $\tilde{Q}_\Delta$  and also for  $Q_\Delta$ .

We close the example with a discussion on an alternative encoding: Instead of  $\tilde{R}(A_1, A_2, B_1, B_2)$ , we can use its lossless decomposition into  $\tilde{R}_A(Id, A_1, A_2)$  and  $\tilde{R}_B(Id, B_1, B_2)$ . Here, a tuple  $(i, a_1, a_2) \in \tilde{R}_A$  encodes that the  $[A]$ -interval in the tuple of  $R$  with identifier  $i$  is mapped to the bitstring  $a_1 \circ a_2$  in the segment tree for  $[A]$ . We thus avoid the explicit materialisation of all combinations of encodings for  $[A]$  and  $[B]$  (and the same for the other two pairs of variables). This decomposition applies systematically to all constructed relations. Each of these new relations has size  $O(N \log N)$ , which is less than  $O(N \log^2 N)$  in our default encoding. In general, for each



**Figure 2: Hypertree decompositions for five of the eight EJ queries in  $\tilde{Q}_\Delta$ . All decompositions have a bag  $\{A_1, B_1, C_1\}$ , whose materialisation requires the computation of a triangle query with equality joins. The remaining three EJ queries admit a decomposition similar with the first one.**

$m$ -way join interval variable, this encoding creates  $m$  new relations regardless of whether such variables occur in the same relational atom in the query. In contrast, our default encoding creates  $m^k$  new relations for each atom that contains  $k$  such  $m$ -way join variables. Although more space efficient, this encoding comes with the same data complexity (modulo log factors) as the one used in the paper.

## 2 RELATED WORK

Algorithms for intersection joins have been developed in the context of temporal [11] and spatial databases [13, 18]. In temporal databases, tuples can be associated with intervals that represent the valid time periods. Temporal or interval joins are used to match tuples that are valid at the same time. In spatial databases, tuples can be associated with 2D objects that are approximated by two intervals defining minimum bounding rectangles [19]. Spatial joins are used to find tuples with overlapping bounding rectangles. Temporal and spatial joins are thus intersection joins. Similarity joins under different distance metrics can be reduced to geometric containment, which is expressible using intersection joins [12].

**Intersection joins.** There is a wealth of work on algorithms for intersection joins, mostly binary joins computed one at a time and over relations with 1D or 2D intervals [18]. These algorithms use indices or partitioning and are typically disk-based with the objective of minimising I/O accesses. Examples of index-based algorithms include: the slot index spatial join [20], the seeded tree join [16], the R-tree join [8], and relational interval tree join [9]. Extensions of binary joins [8] to multi-way joins have also been considered [19]. Partition-based algorithms include: the partition based spatial-merge join [24], the spatial hash join [17], the size separation spatial join [15], the sweeping-based spatial join [5], and the plane-sweep method [27]. The partition-based algorithms can be naturally parallelised and distributed [6, 26, 28]. These algorithms

can compute two-way intersection joins in  $O(N \log N + \text{OUT})$ , where  $\text{OUT}$  is the output size and  $N$  is the input size. To sum up, there is no development on optimal algorithms for queries with intersection joins. Furthermore, most existing approaches focus on one join at a time, which can be suboptimal since they can produce intermediate results asymptotically larger than the final result, as in the case of equality joins [23]. Our approach escapes the limitation of existing intersection join algorithms and benefits from worst-case optimal algorithms for equality joins.

**Inequality joins.** An intersection join can be expressed as a disjunction of inequality joins: Given two intervals  $[l_1, r_1]$  and  $[l_2, r_2]$  defined by their starting and ending points, checking whether they intersect can be expressed as  $(l_1 \leq l_2 \leq r_1) \vee (l_2 < l_1 \leq r_2)$ . IJ queries can thus be reformulated in the framework of Functional Aggregate Queries with Additive Inequalities (FAQ-AI) [2]. The hypergraph of an FAQ-AI has two types of hyperedges: normal hyperedges, one per relation in the query and that covers the nodes representing the variables of that relation, and relaxed hyperedges, one per inequality join and that covers the variables in the inequality. This hypergraph is subject to relaxed hypertree decompositions, which are fractional hypertree decompositions [21] where each normal hyperedge is covered by one bag and each relaxed hyperedge is covered by two adjacent bags. For a database of size  $N$ , an FAQ-AI can be solved in time  $O(N^{\text{subw}_\ell} \text{polylog } N)$ , where  $\text{subw}_\ell$  is the relaxed submodular width of the FAQ-AI and corresponds to the submodular width of the FAQ-AI hypergraph computed over its possible relaxed hypertree decompositions [2]. For the triangle IJ query  $Q_\Delta$  in Section 1.1, the ij-width is  $\text{ijw}(Q_\Delta) = 3/2$  yet  $\text{subw}_\ell(Q_\Delta) = 2$  (Appendix E). Furthermore, it can be shown that  $\text{ijw}$  is lower than  $\text{subw}_\ell$  for the Loomis-Whitney 4, and the 4-clique IJ queries (see Table 1 for a summary and [14] for a full analysis).

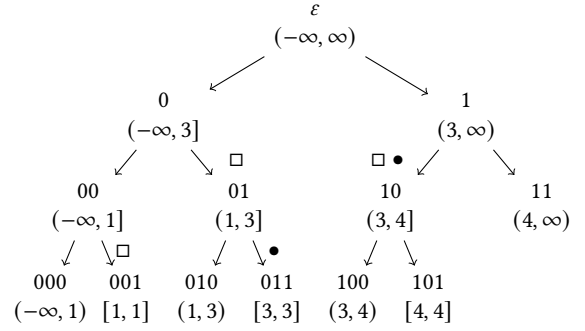
### 3 PRELIMINARIES

This section introduces notation used in the main body of the paper. For lack of space, further preliminaries and proofs are deferred to Appendix A and the full version of the paper [14].

**Segment Tree.** Let  $I$  be a set of  $n$  intervals. Let  $p_1, \dots, p_m$  be the sequence of the distinct endpoints of the intervals in ascending order (so,  $m \leq 2n$ ). Consider the following disjoint intervals called elementary segments that form a partition of the real line:  $(-\infty, p_1)$ ,  $[p_1, p_1]$ ,  $(p_1, p_2)$ ,  $[p_2, p_2]$ ,  $\dots$ ,  $(p_{m-1}, p_m)$ ,  $[p_m, p_m]$ ,  $(p_m, +\infty)$ . The *segment tree*  $\mathfrak{T}_I$  for  $I$  is a complete binary tree<sup>1</sup>, where:

- The leaves of  $\mathfrak{T}_I$  correspond to the elementary segments induced by an order of the endpoints of the intervals in  $I$ : the leftmost leaf corresponds to the leftmost elementary segment, and so on. The elementary segment corresponding to a leaf  $v$  is denoted by  $\text{seg}(v)$ .
- The internal nodes of  $\mathfrak{T}_I$  correspond to segments that are the union of elementary segments at the leaves of their subtrees: the segment  $\text{seg}(u)$  corresponding to an internal node  $u$  is the union of the elementary segments  $\text{seg}(v)$  at the leaves  $v$  in the subtree rooted at  $u$ ;  $\text{seg}(u)$  is thus the union of the segments at its two children.

<sup>1</sup>In a complete binary tree, every level, except possibly the last, is completely filled and the nodes in the last level are positioned as far left as possible. Every node of the segment tree is thus either a leaf or an internal node with exactly two children.



**Figure 3: Segment tree on the set of intervals  $I = \{\square = [1, 4], \bullet = [3, 4]\}$ . The interval  $[1, 4]$  is contained in the canonical subsets of the nodes 001, 01, and 10. The interval  $[3, 4]$  is contained in the canonical subsets of the nodes 011 and 10.**

- Each node  $v$  is associated with the *canonical subset* of  $v$  defined by  $\mathcal{I}_v := \{i \in I \mid \text{seg}(v) \subseteq i \wedge \text{seg}(\text{parent}(v)) \not\subseteq i\}$ . That is, each interval  $i \in I$  is stored into all the maximal segment tree nodes  $v$  with respect to the  $\text{seg}(v)$  inclusion order (or, equivalently, the nodes as high as possible in the tree) such that  $\text{seg}(v) \subseteq i$ .
- Each node of a segment tree is uniquely identified by a bitstring. The root is the empty bitstring, its left child is the bitstring '0', its right child has the bitstring '1', and so on. Throughout the paper, we are using a node and its corresponding bitstring interchangeably.

Let  $V(\mathfrak{T}_I)$  denote the set of nodes in the segment tree  $\mathfrak{T}_I$ . For a node  $u \in V(\mathfrak{T}_I)$ , let  $\text{anc}(u)$  be the set of ancestors of  $u$  including  $u$ . Given an interval (or point)  $x$ ,  $\text{leaf}(x)$  denotes the leaf that includes the left endpoint of the interval  $x$  (or includes  $x$ , in case  $x$  is a point).

**DEFINITION 3.1 (CANONICAL PARTITION).** Let  $I$  be a set of intervals and  $x \in I$ . The *Canonical Partition* of  $x$  with respect to  $I$  is  $CP_I(x) := \{u \in V(\mathfrak{T}_I) \mid x \in \mathcal{I}_u\}$ .

We use the following properties of a segment tree.

**PROPERTY 3.2 (SEGMENT TREE).** Let  $I$  be any set of intervals and  $\mathfrak{T}_I$  be the segment tree for it.

- (1) Let  $u$  and  $v$  be nodes in the segment tree. Then  $u \in \text{anc}(v)$  if and only if  $\text{seg}(u) \supseteq \text{seg}(v)$ . Equivalently,  $u$  is a prefix of  $v$ .
- (2) For any interval  $x \in I$ , there cannot be two nodes in  $CP_I(x)$  such that one of them is an ancestor of the other.
- (3) For any interval  $x \in I$ ,  $CP_I(x)$  has size and can be computed in time  $O(\log |I|)$ .

Since  $\mathfrak{T}_I$  is a complete binary tree with  $O(|I|)$  leaves, the size of the tree is  $O(|I|)$ , while its height is  $O(\log |I|)$ . By Property 3.2(3), it follows that  $\mathfrak{T}_I$  has size  $O(|I| \cdot \log |I|)$  [27]. Figure 3 gives an example segment tree for a set of two intervals.

**Queries.** We consider queries with intersection joins and equality joins. An intersection join is expressed using an *interval variable*, denoted by  $[X]$ , which takes as values intervals with real-valued endpoints from a finite domain  $\text{Dom}([X])$ . An equality join is expressed using a *point variable* or variable for short, denoted by  $X$ ,

whose values are real numbers from a finite domain  $\text{Dom}(X)$ . For an interval  $x$ , we use  $x.l$  and  $x.r$  to denote its left and respectively right endpoints. Given a set  $e$  of variables,  $R_e \subseteq \prod_{X \in e} \text{Dom}(X)$  is a relation consisting of tuples of  $|e|$  real values; relations over intervals are defined similarly by replacing  $X$  with  $[X]$ . A tuple  $t$  with schema  $e$  is a mapping of the variables in  $e$  to values in their domains. We denote by  $t(X)$  the value for variable  $X$  (or  $[X]$ ) in  $t$  and by  $t(e')$  the set mapping variables in  $e' \subseteq e$  to their values in  $t$ .

A (multi-)hypergraph  $\mathcal{H} = (\mathcal{V}, \mathcal{E})$  has a set  $\mathcal{V}$  of vertices and a multiset  $\mathcal{E} \subseteq 2^{\mathcal{V}}$  of hyperedges. We label the hyperedges to distinguish between those representing the same set of vertices. For a vertex  $X \in \mathcal{V}$ ,  $\mathcal{E}_X$  denotes the subset of  $\mathcal{E}$  that contains  $X$ .

**DEFINITION 3.3 (QUERIES).** *Given a hypergraph  $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ , where  $\mathcal{V}$  is a set of variables, a query over  $\mathcal{H}$  has the form  $Q = \bigwedge_{e \in \mathcal{E}} R_e(e)$ . If the vertices in  $\mathcal{H}$  are interval variables, then  $Q$  is a Boolean conjunctive query with intersection joins, or IJ for short. If the vertices in  $\mathcal{H}$  are point variables, then  $Q$  is a Boolean conjunctive query with equality joins, or EJ for short. If the vertices in  $\mathcal{H}$  are point and interval variables, then  $Q$  is a Boolean conjunctive query with intersection and equality joins, or EIJ for short.*

An EIJ  $Q = \bigwedge_{e \in \mathcal{E}} R_e(e)$  evaluates to true if and only if there exist tuples  $(t_e)_{e \in \mathcal{E}} \in \prod_{e \in \mathcal{E}} R_e$  that satisfy the following:

- $\forall [X] \in \mathcal{V}$ , we have  $(\bigcap_{e \in \mathcal{E}_{[X]}} t_e([X])) \neq \emptyset$ .
- $\forall X \in \mathcal{V}$ ,  $t_e(X)$  is the same for all  $e \in \mathcal{E}_X$ .

For the evaluation of an IJ query  $Q$  with hypergraph  $\mathcal{H} = (\mathcal{V}, \mathcal{E})$  over a database  $\mathbf{D}$ , we assume without loss of generality that the schema of  $\mathbf{D}$  is given by  $\mathcal{H}$ ; any database can be brought into this form by appropriately ensuring a bijection between the vertices in  $\mathcal{V}$  and attributes in  $\mathbf{D}$  and a bijection between the hyperedges  $e \in \mathcal{E}$  and the relations  $R_e$  over schema  $e$  in  $\mathbf{D}$ .

Given a set  $S$ , a permutation of  $S$  is an ordered sequence of the elements in  $S$ . We denote by  $\pi(S)$  the set of all permutations of the elements in  $S$ . For a sequence  $s$ ,  $s_i$  denotes its  $i$ -th element. The concatenation of sequences  $s_1, \dots, s_k$  is denoted by  $s_1 \circ \dots \circ s_k$ .

## 4 FROM INTERSECTIONS TO EQUALITIES

In this section, we show that the IJ evaluation problem can be reduced to the EJ evaluation problem. This forward reduction is used to give an upper bound on the time complexity for the former problem using the complexity of the latter problem. Section 5 then presents a backward reduction to give a corresponding lower bound on the time complexity of the IJ evaluation problem (recall Figure 1).

### 4.1 Rewriting the Intersection Predicate

At the core of IJ evaluation lies the non-emptiness check of the intersection of  $k$  intervals  $x_1, \dots, x_k \in \mathcal{I}$ :  $(\bigcap_{i \in [k]} x_i) \neq \emptyset$ . We call this check the *intersection predicate*. In this section, we show how to rewrite this predicate into an equivalent form that uses the canonical partitions of the intervals in a segment tree  $\mathcal{T}_I$ .

Since the elementary segments that correspond to the leaves of  $\mathcal{T}_I$  form a partition of  $\mathbb{R}$ , for any point  $p \in \mathbb{R}$  there is precisely one leaf node  $\text{leaf}(p)$  such that  $p \in \text{seg}(\text{leaf}(p))$ . By Property 3.2(1),  $\text{anc}(\text{leaf}(p)) = \{v \in V(\mathcal{T}_I) \mid p \in \text{seg}(v)\}$ . That is, the nodes whose segments contain the point  $p$  are precisely the ancestors of  $\text{leaf}(p)$ .

**LEMMA 4.1 (INTERSECTION PREDICATE REWRITING 1).** *For any set of intervals  $S = \{x_1, \dots, x_k\} \subseteq \mathcal{I}$ , the predicate  $(\bigcap_{i \in [k]} x_i) \neq \emptyset$  is equivalent to:*

$$\bigvee_{i \in [k]} \left[ \bigvee_{(v_1, \dots, v_k) \in \text{anc}(\text{leaf}(x_i))^k} \left( \bigwedge_{\substack{j \in [k] \\ j \neq i}} v_j \in CP_I(x_j) \right) \right]$$

Lemma 4.1 states the following. The intervals in  $S$  intersect if and only if there is an interval  $x_i \in S$  such that the canonical partitions of each other interval in  $S$  contain an ancestor of  $\text{leaf}(x_i)$ . By construction, this leaf contains the left endpoint of  $x_i$ .

**PROPERTY 4.2.** *Consider a set of intervals  $S = \{x_1, \dots, x_k\} \subseteq \mathcal{I}$  and a segment tree  $\mathcal{T}_I$ . For any  $x_i \in S$ , there can be at most one tuple of nodes  $v_j \in \text{anc}(\text{leaf}(x_i))$  for  $j \in [k]$ ,  $j \neq i$  that satisfy the conjunction of Lemma 4.1.*

The conjunction in Lemma 4.1 can be satisfied by several  $i$ -values when there are several intervals in  $S$  that have the same left endpoint. The database can be transformed such that any two intervals from different relations have distinct left endpoints without affecting query evaluation [14]. If the intervals in  $S$  have distinct left endpoints, then the conjunction in Lemma 4.1 can be satisfied by at most one  $i \in [k]$ , namely the one with the maximum left endpoint of the intervals in  $S$ ; this is also the left endpoint of the interval representing the intersection of all intervals in  $S$ .

Consider a path from the root of the segment tree  $\mathcal{T}_I$  down to  $\text{leaf}(x_i)$  from Lemma 4.1. The nodes  $v_1, \dots, v_k$  from Lemma 4.1 all lie on this path. These nodes satisfy  $v_j \in CP_I(x_j)$  for all  $j \in [k] - \{i\}$ . WLOG let  $v_i = \text{leaf}(x_i)$ . Let  $u_1, \dots, u_k$  be a permutation of  $v_1, \dots, v_k$  listing them in order along the path where  $u_j \in \text{anc}(u_{j+1})$  for all  $j \in [k-1]$  and  $u_k = v_i = \text{leaf}(x_i)$ . Let  $\sigma_1, \dots, \sigma_k$  be the corresponding permutation of the line segments  $x_1, \dots, x_k$  where  $u_j \in CP_I(\sigma_j)$  for all  $j \in [k-1]$  and  $\sigma_k = x_i$ . Such a permutation always exists. Hence it is possible to reformulate Lemma 4.1 where on the right-hand side we consider (a disjunction over) all such permutations  $\sigma_1, \dots, \sigma_k$  (subsuming the disjunction over  $i \in [k]$ ) and in return we get to assume that the inner disjunction is now over *ordered*  $(u_1, \dots, u_k) \in \text{anc}(\text{leaf}(\sigma_k))^k$ , meaning that  $u_j \in \text{anc}(u_{j+1})$  for all  $j \in [k-1]$  and  $u_k = \text{leaf}(\sigma_k)$ . This leads to the following variant of Lemma 4.1, which will be easier to utilize later in our reduction.

**LEMMA 4.3 (INTERSECTION PREDICATE REWRITING 2).** *For any set of intervals  $S = \{x_1, \dots, x_k\} \subseteq \mathcal{I}$ , the predicate  $(\bigcap_{i \in [k]} x_i) \neq \emptyset$  is equivalent to:*

$$\bigvee_{\sigma \in \pi(S)} \left[ \bigvee_{\substack{(u_1, \dots, u_k) \in \text{anc}(\text{leaf}(\sigma_k))^k \\ \forall j \in [k-1]: u_j \in \text{anc}(u_{j+1}) \\ u_k = \text{leaf}(\sigma_k)}} \left( \bigwedge_{j \in [k-1]} u_j \in CP_I(\sigma_j) \right) \right]$$

By Property 4.2, for a permutation  $\sigma \in \pi(S)$ , there can be at most one tuple  $(u_j)_{j \in [k-1]}$  that satisfies the conjunction. However, even if the intervals in  $S$  have distinct left endpoints, the predicate of Lemma 4.3 may be satisfied by multiple permutations. To see this, suppose that  $\sigma$  and  $(u_j)_{j \in [k-1]}$  satisfy the predicate. If there is  $j \in [k-1]$  such that  $u_j = u_{j+1}$ , then the permutation  $\sigma'$ , obtained

by swapping  $\sigma_j$  and  $\sigma_{j+1}$  in  $\sigma$ , together with the tuple  $(u_j)_{j \in [k-1]}$ , also satisfy the predicate. It is possible to further restrict the permutations such that each tuple of segment tree nodes that satisfies the conjunction corresponds to exactly one permutation [14]. The equivalence in Lemma 4.3 can be alternatively expressed using the bitstrings of the nodes in the segment tree. By Property 3.2 (1) the expression  $u_j \in \text{anc}(u_{j+1})$  can be equivalently stated as  $u_j$  being a prefix of  $u_{j+1}$ . In other words, there exists a tuple of bitstrings  $(b_1, \dots, b_k)$  such that  $u_j = b_1 \circ \dots \circ b_j$  for  $j \in [k]$ . This observation leads to the following rewrite of Lemma 4.3.

LEMMA 4.4 (INTERSECTION PREDICATE REWRITING 3). *Consider a set of intervals  $S = \{x_1, \dots, x_k\} \subseteq \mathcal{I}$ . The predicate  $(\bigcap_{x \in S} x) \neq \emptyset$  is true if and only if there exists a permutation  $\sigma \in \pi(S)$  and a tuple of bitstrings  $(b_1, \dots, b_k)$  such that:*

- $(b_1 \circ \dots \circ b_j) \in CP_{\mathcal{I}}(\sigma_j)$  for  $j \in [k-1]$ , and
- $(b_1 \circ \dots \circ b_j) = \text{leaf}(\sigma_j)$  for  $j = k$ .

Section 4.2 lifts the rewriting in Lemma 4.4 to the level of queries.

## 4.2 One-Step Forward Reduction

For an IJ query  $Q$  with hypergraph  $\mathcal{H} = (\mathcal{V}, \mathcal{E})$  and a database  $\mathbf{D}$ , the forward reduction proceeds iteratively on  $Q$  and  $\mathbf{D}$  and resolves one join interval variable at a time. Let this variable be  $[X]$ . The reduction yields a new query  $Q_{[X]}$  and a new database  $\mathbf{D}_{[X]}$  such that  $Q_{[X]}(\mathbf{D}_{[X]})$  is true if and only if  $Q(\mathbf{D})$  is true.

The core computation needed to evaluate  $Q$  over  $\mathbf{D}$  is the intersection predicate  $(\bigcap_{x \in S} x) \neq \emptyset$ , where  $S$  consists of one input interval per relation involved in the intersection join on  $[X]$ . Lemma 4.4 explains how to express this computation for any subset  $S$  of an input set of intervals  $\mathcal{I}$  for  $[X]$  using the segment tree  $\mathcal{I}_T$  for  $\mathcal{I}$ .

Let  $k = |\mathcal{E}_{[X]}|$  be the number of hyperedges in  $\mathcal{H}$  containing  $[X]$ . The reduction maps  $[X]$  to fresh point variables  $X_1, \dots, X_k$  that range over the possible bitstrings of the segment tree nodes from the canonical partitions of the intervals of  $[X]$ .

Given a permutation  $\sigma = (\sigma_1, \dots, \sigma_k) \in \pi(\mathcal{E}_{[X]})$  of the hyperedges  $\mathcal{E}_{[X]}$  containing  $[X]$ , each hyperedge  $\sigma_i$  induces a fresh hyperedge  $\tilde{\sigma}_i$  that has the fresh point variables  $X_1, \dots, X_i$  in place of the original interval variable  $[X]$ :  $\tilde{\sigma}_i = \sigma_i \setminus \{[X]\} \cup \{X_1, \dots, X_i\}$ .

DEFINITION 4.5 (ONE-STEP HYPERGRAPH TRANSFORMATION). *Given a hypergraph  $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ , an interval variable  $[X]$ , and any permutation  $\sigma \in \pi(\mathcal{E}_{[X]})$ , the hypergraph  $\tilde{\mathcal{H}}_{([X], \sigma)}$  has the set  $\mathcal{V}_{[X]}$  of vertices and the set  $\mathcal{E}_{([X], \sigma)}$  of hyperedges, where  $\mathcal{V}_{[X]} = \mathcal{V} \setminus \{[X]\} \cup \{X_1, \dots, X_k\}$  and  $\mathcal{E}_{([X], \sigma)} = \mathcal{E} \setminus \{\sigma_i \mid i \in [k]\} \cup \{\tilde{\sigma}_i \mid i \in [k]\}$ . The set  $\tilde{\mathcal{H}}_{[X]} = \{\tilde{\mathcal{H}}_{([X], \sigma)} \mid \sigma \in \pi(\mathcal{E}_{[X]})\}$  consists of all hypergraphs created from  $\mathcal{H}$  by resolving the interval variable  $[X]$ .*

For a given permutation  $\sigma$ , there is a one-to-one correspondence between the hyperedges in  $\mathcal{H}$  and those in  $\tilde{\mathcal{H}}_{([X], \sigma)}$ . We obtain as many new hypergraphs as the number of permutations of  $\mathcal{E}_{[X]}$ .

EXAMPLE 4.6. Let  $\mathcal{H}$  be a hypergraph with vertices  $\{[A], [B], [C]\}$  and edges  $e_1 = e_2 = \{[A], [B], [C]\}$  and  $e_3 = \{[A]\}$ .

We reduce  $\mathcal{H}$  by resolving the interval variable  $[A]$ . Since  $[A]$  occurs in three edges, we create three point variables  $A_1, A_2, A_3$  and consider six permutations. The new edges created for the permutation  $\sigma = (e_1, e_2, e_3)$  are:  $\tilde{\sigma}_1 = \{A_1, [B], [C]\}$ ,  $\tilde{\sigma}_2 = \{A_1, A_2, [B], [C]\}$ ,

and  $\tilde{\sigma}_3 = \{A_1, A_2, A_3\}$ . For the permutation  $(e_3, e_2, e_1)$ , the new edges are:  $\{A_1\}$ ,  $\{A_1, A_2, [B], [C]\}$ , and  $\{A_1, A_2, A_3, [B], [C]\}$ .  $\square$

Each hypergraph  $\tilde{\mathcal{H}}_{([X], \sigma)}$  defines a new query  $Q_{([X], \sigma)}$  and the corresponding database  $\mathbf{D}_{([X], \sigma)}$ . We next explain how to construct the new query and the new database.

The query  $Q$  is rewritten according to the new hypergraphs: For each permutation  $\sigma$ , we create an IJ query  $\tilde{Q}_{([X], \sigma)}$ , which has equality joins and possibly remaining intersection joins, whose hypergraph is  $\tilde{\mathcal{H}}_{([X], \sigma)}$ . By taking all permutations, we thus create a query  $\tilde{Q}_{[X]}$  that is a disjunction of IJ queries such that each such query has one join interval variable less, namely  $[X]$ . Note that  $Q$  need not be an IJ query: It may have both intersection and equality joins, for instance if it is the result of a previous rewriting step.

DEFINITION 4.7 (ONE-STEP QUERY REWRITING). *For any IJ query  $Q$  with hypergraph  $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ , the IJ query  $\tilde{Q}_{([X], \sigma)}$  with hypergraph  $\tilde{\mathcal{H}}_{([X], \sigma)} = (\mathcal{V}_{[X]}, \mathcal{E}_{([X], \sigma)})$  is defined by:*

$$\tilde{Q}_{([X], \sigma)} := \bigwedge_{\tilde{e} \in \tilde{\mathcal{E}}_{([X], \sigma)}} \tilde{R}(\tilde{e})$$

*The query  $\tilde{Q}_{[X]}$  is the disjunction of the IJ queries  $\tilde{Q}_{([X], \sigma)}$  over all possible permutations  $\sigma \in \pi(\mathcal{E}_{[X]})$ :*

$$\tilde{Q}_{[X]} := \bigvee_{\sigma \in \pi(\mathcal{E}_{[X]})} \tilde{Q}_{([X], \sigma)}.$$

EXAMPLE 4.8. The IJ query  $Q = R([A], [B], [C]) \wedge S([A], [B], [C]) \wedge T([A])$  has the hypergraph  $\mathcal{H}$  in Example 4.6. The permutation  $(e_1, e_2, e_3)$  yields  $\tilde{Q}_1 = \tilde{R}(A_1, [B], [C]), \tilde{S}(A_1, A_2, [B], [C]), \tilde{T}(A_1, A_2, A_3)$ . The permutation  $(e_3, e_2, e_1)$  yields  $\tilde{Q}_2 = \tilde{R}(A_1, A_2, A_3, [B], [C]), \tilde{S}(A_1, A_2, [B], [C]), \tilde{T}(A_1)$ . The final query  $\tilde{Q}_{[A]}$  is a disjunction of six IJ queries, including  $\tilde{Q}_1$  and  $\tilde{Q}_2$ .  $\square$

For each new hyperedge  $\tilde{\sigma}_i$  in  $\tilde{\mathcal{H}}_{([X], \sigma)}$ , there is a new relation  $\tilde{R}_{\tilde{\sigma}_i}(\tilde{\sigma}_i)$ . To avoid clutter, we denote it by  $\tilde{R}(\tilde{\sigma}_i)$ ; its schema  $\tilde{\sigma}_i$  uniquely identifies the transformation of the original relation  $R$ .

DEFINITION 4.9 (ONE-STEP DATABASE TRANSFORMATION). *The database  $\tilde{\mathbf{D}}_{([X], \sigma)}$  is constructed from the database  $\mathbf{D}$  as follows. For each tuple  $t \in R(\sigma_i)$ , we construct tuples  $\tilde{t} \in \tilde{R}(\tilde{\sigma}_i)$  such that:*

- $t(\sigma_i \setminus \{[X]\}) = \tilde{t}(\tilde{\sigma}_i \setminus \{X_1, \dots, X_i\})$
- $(\tilde{t}(X_1) \circ \dots \circ \tilde{t}(X_i)) \in CP_{\mathcal{I}}(t([X]))$  if  $i \in [k-1]$
- $(\tilde{t}(X_1) \circ \dots \circ \tilde{t}(X_i)) = \text{leaf}(t([X]))$  if  $i = k$

*The relations whose schemas do not contain  $[X]$  are copied from  $\mathbf{D}$  to  $\tilde{\mathbf{D}}_{([X], \sigma)}$ . The new database  $\tilde{\mathbf{D}}_{[X]}$  is the set of all relations in the databases  $\tilde{\mathbf{D}}_{([X], \sigma)}$ .*

The number of tuples  $\tilde{t}$  constructed for a tuple  $t$  in Definition 4.9 depends on the size of the canonical partition  $CP_{\mathcal{I}}(t([X]))$  of  $t([X])$  and on the number of ways we can partition the bitstring of a node in the canonical partition into  $i$  substrings. Overall, this number is poly-logarithmic in the number of input intervals  $\mathcal{I}$ . This is made more precise in the next lemma.

LEMMA 4.10. *Each new relation  $\tilde{R}(\tilde{\sigma}_i)$  in database  $\tilde{\mathbf{D}}_{([X], \sigma)}$  constructed from the database  $\mathbf{D}$  following Definition 4.9 has the size:  $O(|R(\sigma_i)| \cdot \log^i |\mathcal{I}|)$  if  $i \in [k-1]$ ; and  $O(|R(\sigma_i)| \cdot \log^{i-1} |\mathcal{I}|)$  if  $i = k$ . It can be constructed in time proportional to its size.*

The transformations in Definitions 4.9 and 4.7 preserve the equivalence to the original evaluation problem: The result of  $Q$  over  $\mathbf{D}$  is the same as the result of  $\tilde{Q}_{[X]}$  over  $\tilde{\mathbf{D}}_{[X]}$ .

LEMMA 4.11. *Given any IJ query  $Q$ , interval variable  $[X]$  in  $Q$ , and any database  $\mathbf{D}$ , let the IJ query  $\tilde{Q}_{[X]}$  and database  $\tilde{\mathbf{D}}_{[X]}$  be constructed as per Definitions 4.9 and 4.7. Then,  $Q(\mathbf{D})$  is true if and only if  $\tilde{Q}_{[X]}(\tilde{\mathbf{D}}_{[X]})$  is true.*

EXAMPLE 4.12. We demonstrate the database transformation for the query  $Q_\Delta$  in Section 1.1. Consider the interval variable  $[A]$ . We convert  $Q_\Delta$  into the disjunction of two queries with two new point variables  $A_1$  and  $A_2$  in lieu of the interval variable  $[A]$ :

$$\begin{aligned}\tilde{Q}' &= \tilde{R}_1(A_1, [B]) \wedge S([B], [C]) \wedge \tilde{T}_1(A_1, A_2, [C]), \\ \tilde{Q}'' &= \tilde{R}_2(A_1, A_2, [B]) \wedge S([B], [C]) \wedge \tilde{T}_2(A_1, [C]).\end{aligned}$$

Let  $\mathbf{D}$  be the input database with relations  $R, S$  and  $T$  and let  $N$  be the size of  $\mathbf{D}$ . By Definition 4.9, we construct a new database instance  $\tilde{\mathbf{D}}_{[A]}$  that consists of  $S$  and four new relations  $\tilde{R}_1, \tilde{T}_1, \tilde{R}_2$  and  $\tilde{T}_2$ . By Lemma 4.11,  $Q_\Delta(\mathbf{D})$  is true if and only if  $\tilde{Q}'(\tilde{\mathbf{D}}_{[A]})$  or  $\tilde{Q}''(\tilde{\mathbf{D}}_{[A]})$  are true.

Let  $\mathcal{I}$  be the set of all  $[A]$ -intervals in  $R$  and  $T$ . We have  $|\mathcal{I}| \leq 2 \cdot N$ . Let  $\mathcal{E}$  be the set of endpoints of these intervals. We have  $|\mathcal{E}| \leq 2 \cdot |\mathcal{I}|$ . Assume that  $\mathcal{E} = \{0, 1, 2, \dots, k\}$  for some integer  $k \leq 2 \cdot N$ . This is without loss of generality since the intersection problem does not depend on the absolute values of the end points but only on their relative positioning. Assume also that  $k$  is a power of 2; otherwise, replace  $k$  with the smallest power of 2 that is  $\geq k$ . Let  $\mathfrak{T}_\mathcal{I}$  be the segment tree whose root corresponds to the interval  $[0, k]$ , and the left and right children of each node correspond to the left and right halves of the corresponding interval respectively. Assume without loss of generality that each interval  $i \in \mathcal{I}$  corresponds to a node in  $\mathfrak{T}_\mathcal{I}$ ; otherwise,  $i$  can be broken down into  $O(\log_2 k) = O(\log_2 N)$  intervals that correspond to nodes in  $\mathfrak{T}_\mathcal{I}$  (Property 3.2 (3)). Each node  $n$  in  $\mathfrak{T}_\mathcal{I}$  can be encoded as a binary string, cf. Section 3: If  $n$  corresponds to the string  $b$ , then its left and right children correspond to the strings “ $b0$ ” and “ $b1$ ” respectively. For an interval  $i$  that corresponds to  $n$  in  $\mathfrak{T}_\mathcal{I}$ , let  $\text{bin}(i)$  denote the binary string that encodes node  $n$ . Let  $i_1$  and  $i_2$  be two intervals that correspond to the nodes  $n_1$  and  $n_2$ . They intersect if and only if one of the two nodes is an ancestor of the other (or the nodes are the same). That is, one of them contains the other (Property 3.3, part (1)). Equivalently, they intersect if and only if one of the two binary strings  $\text{bin}(i_1)$  and  $\text{bin}(i_2)$  is a prefix of the other (Property 3.2 (1)).

Define the new relations as follows (Definition 4.9):

$$\begin{aligned}\tilde{R}_1 &= \{(\text{bin}([a]), [b]) \mid ([a], [b]) \in R\} \\ \tilde{T}_2 &= \{(\text{bin}([a]), [c]) \mid ([a], [c]) \in T\} \\ \tilde{T}_1 &= \{(a_1, a_2, [c]) \mid ([a], [c]) \in T \wedge a_1 \circ a_2 = \text{bin}([a])\} \\ \tilde{R}_2 &= \{(a_1, a_2, [b]) \mid ([a], [b]) \in R \wedge a_1 \circ a_2 = \text{bin}([a])\}.\end{aligned}$$

By Lemma 4.10 and the assumption that each  $[A]$ -interval corresponds to exactly one node of  $\mathfrak{T}_\mathcal{I}$ , we have  $|\tilde{R}_1| = |R|$ ,  $|\tilde{T}_2| = |T|$ , and both  $\tilde{R}_1$  and  $\tilde{T}_2$  can be constructed in linear time. Furthermore,  $|\tilde{T}_1| = O(|T| \cdot \log N)$  because there are  $O(\log N)$  ways to break a binary string  $\text{bin}([a])$  of length  $O(\log N)$  in two.  $\tilde{T}_1$  can be constructed in time  $O(|T| \cdot \log N)$ . Similarly for  $\tilde{R}_2$  (Lemma 4.10).

Then,  $\tilde{Q}'(\tilde{\mathbf{D}}_{[A]})$  holds if and only if  $Q(\mathbf{D})$  has a satisfying assignment where the  $[A]$ -interval from  $R$  contains the  $[A]$ -interval from  $T$ . Also,  $\tilde{Q}''(\tilde{\mathbf{D}}_{[A]})$  holds if and only if  $Q(\mathbf{D})$  has a satisfying assignment where the  $[A]$ -interval from  $R$  is contained in the  $[A]$ -interval from  $T$ . Consequently  $Q(\mathbf{D})$  holds if and only if  $\tilde{Q}'(\tilde{\mathbf{D}}_{[A]})$  or  $\tilde{Q}''(\tilde{\mathbf{D}}_{[A]})$  holds (Lemma 4.11).  $\square$

### 4.3 Full Forward Reduction

In this section we show how to completely reduce (1) any IJ query to a disjunction of EJ queries and (2) any database to a database with bitstrings in place of intervals for join interval variables. In particular, the full reduction is obtained by iteratively applying the reduction step from Section 4.2 for each interval variable to the result of the previous reduction step or to the input query and database in case of the first reduction step.

---

#### Algorithm 1 IJ to EJ Reduction

---

**Input:** IJ query  $Q$  with hypergraph  $\mathcal{H}$ , database  $\mathbf{D}$

---

```

1: procedure REDUCE( $\tilde{\mathbf{H}} = \{\mathcal{H}\}, Q = \{Q\}, \tilde{\mathbf{D}} = \mathbf{D}$ )
2:   for each interval join variable  $[X]$  in  $Q$  do
3:      $\tilde{\mathbf{H}}_0 := \tilde{\mathbf{H}}; \tilde{\mathbf{H}} := \emptyset$ 
4:     for each  $\mathcal{H} \in \tilde{\mathbf{H}}_0$  do
5:       create  $\tilde{\mathcal{H}}_{[X]}$  from  $\mathcal{H}$  following Definition 4.5
6:        $\tilde{\mathbf{H}} := \tilde{\mathbf{H}} \cup \tilde{\mathcal{H}}_{[X]}$ 
7:      $\tilde{Q}_0 := Q; \tilde{Q} := \emptyset$ 
8:     for each  $Q \in \tilde{Q}_0$  do
9:       for each  $\sigma \in \pi(\mathcal{E}_{[X]})$  do
10:        create  $\tilde{Q}_{([X], \sigma)}$  from  $Q$  following Definition 4.7
11:         $\tilde{Q} := \tilde{Q} \cup \{\tilde{Q}_{([X], \sigma)}\}$ 
12:       create  $\tilde{\mathbf{D}}_{[X]}$  from  $\tilde{\mathbf{D}}$  following Definition 4.9
13:        $\tilde{\mathbf{D}} := \tilde{\mathbf{D}}_{[X]}$ 
14:   return  $(\tilde{\mathbf{H}}, \tilde{Q}, \tilde{\mathbf{D}})$ 
```

---

Algorithm 1 details the reduction. The result is a triple consisting of: the set  $\tilde{\mathbf{H}}$  of hypergraphs constructed by iteratively resolving the join interval variables in the input IJ query  $Q$ ; the set  $\tilde{Q}$  of EJ queries, with one such query per hypergraph in  $\tilde{\mathbf{H}}$ ; and the database  $\tilde{\mathbf{D}}$ . The final query is the disjunction of the EJ queries in  $\tilde{Q}$ .

We further define the transformation function  $\tau$  that takes any hypergraph  $\mathcal{H}$  to the set of hypergraphs  $\tilde{\mathbf{H}}$ :  $\tau(\mathcal{H}) = \tilde{\mathbf{H}}$ , where  $(\tilde{\mathbf{H}}, \tilde{Q}, \tilde{\mathbf{D}}) = \text{REDUCE}(\{\mathcal{H}\}, \{Q\}, \mathbf{D})$ . This is used in the following sections to define the complexity of IJ queries.

The next theorem states that our reduction is correct.

THEOREM 4.13 (CORRECTNESS). *For any IJ query  $Q$  with hypergraph  $\mathcal{H}$  and any database  $\mathbf{D}$ , it holds that  $Q(\mathbf{D})$  is true if and only if  $\bigvee_{\tilde{Q} \in \tilde{Q}} \tilde{Q}(\tilde{\mathbf{D}})$  is true, where  $(\tilde{\mathbf{H}}, \tilde{Q}, \tilde{\mathbf{D}}) = \text{REDUCE}(\{\mathcal{H}\}, \{Q\}, \mathbf{D})$ .*

### 4.4 Complexity of IJ Queries

We give the data complexity of IJ queries using the reduction to EJ queries from Section 4.3. We next define a new width measure for IJ queries called the ij-width using the submodular width of the EJ queries obtained in the full reduction (Definition A.9).

| IJ Query         | FAQ-AI [2]                         | Our approach          |
|------------------|------------------------------------|-----------------------|
| Triangle         | $O(N^2 \log^3 N)$                  | $O(N^{3/2} \log^3 N)$ |
| Loomis-Whitney-4 | $O(N^2 \log^k N)$ , for $k \geq 9$ | $O(N^{5/3} \log^8 N)$ |
| 4-clique         | $O(N^3 \log^k N)$ , for $k \geq 5$ | $O(N^2 \log^8 N)$     |

**Table 1: Our approach versus FAQ-AI for three IJ queries.**

DEFINITION 4.14 (IJ-WIDTH). *For any hypergraph  $\mathcal{H}$ , the ij-width of  $\mathcal{H}$  is defined as follows:*

$$\text{ijw}(\mathcal{H}) := \max_{\tilde{\mathcal{H}} \in \tau(\mathcal{H})} \text{subw}(\tilde{\mathcal{H}})$$

The complexity of a given IJ query is that of the most expensive EJ query constructed by the full reduction. This justifies taking the maximum in the definition of the ij-width. The optimality yardstick for the evaluation of EJ queries is given by the submodular width [21] (see also discussion in Section 1), which justifies the use of this width measure in the definition of the ij-width.

Let  $\mathcal{V}' \subseteq \mathcal{V}$  be the set of join interval variables in the query and  $k_{[X]} = |\mathcal{E}_{[X]}|$  be the number of hyperedges containing the interval variable  $[X]$ . Our full reduction constructs up to  $\prod_{[X] \in \mathcal{V}'} k_{[X]}$  new relations and up to  $\prod_{[X] \in \mathcal{V}'} k_{[X]}!$  EJ queries. By Lemma 4.10, each new relation has size  $O(N \text{polylog } N)$ , where  $N$  is the size of the input relations. The number of constructed EJ queries only depends on the structure of  $Q$ .

THEOREM 4.15. *Given any IJ query  $Q$  with hypergraph  $\mathcal{H}$  and database  $\mathbf{D}$ ,  $Q(\mathbf{D})$  can be computed in time  $O(|\mathbf{D}|^{\text{ijw}(\mathcal{H})} \cdot \text{polylog}|\mathbf{D}|)$ .*

Table 1 gives the time complexities for FAQ-AI [2] and our approach for the cyclic IJ queries: triangle, Loomis Whitney with 4 variables, and 4-clique. Appendix E details the FAQ-AI evaluation of a reformulation of the triangle IJ query using inequality joins.

## 5 FROM EQUALITIES TO INTERSECTIONS: REDUCTION OPTIMALITY

In the previous section, we showed how to reduce the evaluation of an IJ query  $Q$  over a database  $\mathbf{D}$  of intervals to a disjunction  $\tilde{Q}$  of EJ queries over a database  $\tilde{\mathbf{D}}$  of numbers, where  $|\tilde{\mathbf{D}}| = O(|\mathbf{D}| \cdot \text{polylog}|\mathbf{D}|)$ . This proves that the runtime on  $Q$  is upper bounded by the maximum upper bound over all queries in  $\tilde{Q}$  (within a polylog factor). In this section, we do the opposite. We show that the runtime on  $Q$  is also lower bounded by the maximum lower bound over all queries in  $\tilde{Q}$ . We start with an EJ query  $\tilde{Q}$ , whose query structure matches that of one of the queries in  $\tilde{Q}$ , and with an arbitrary database  $\tilde{\mathbf{D}}_2$  over the schema of  $\tilde{Q}$ . The values in  $\tilde{\mathbf{D}}_2$  are numbers and can be chosen independently from  $\tilde{\mathbf{D}}$  and  $\mathbf{D}$  in the forward reduction. We show how to reduce  $\tilde{Q}(\tilde{\mathbf{D}}_2)$  to  $Q(\mathbf{D}_2)$ , where  $Q$  is an IJ query, whose hypergraph matches that of the original  $Q$ , and  $\mathbf{D}_2$  is some database with intervals and  $O(|\tilde{\mathbf{D}}_2|)$  size. See Figure 1.

EXAMPLE 5.1. Consider the IJ query  $Q_\Delta$  from Example 1.1. WLOG let's take the EJ query  $\tilde{Q}_3$  that results from the reduction:

$$\tilde{Q}_3 = R_{2,1}(A_1, A_2, B_1) \wedge S_{2,2}(B_1, B_2, C_1, C_2) \wedge T_{1,1}(A_1, C_1)$$

Consider an arbitrary database  $\tilde{\mathbf{D}} = \{\tilde{\mathbf{D}}_{R_{2,1}}, \tilde{\mathbf{D}}_{S_{2,2}}, \tilde{\mathbf{D}}_{T_{1,1}}\}$  over the schema  $\{R_{2,1}, S_{2,2}, T_{1,1}\}$ . We can reduce solving  $\tilde{Q}_3(\tilde{\mathbf{D}})$  to solving

query  $Q_\Delta$  over another database  $\mathbf{D} = \{\mathbf{D}_R, \mathbf{D}_S, \mathbf{D}_T\}$  (whose values are intervals) constructed as follows. Let  $F$  be a function that maps binary strings  $\{0, 1\}^*$  into intervals  $[x, y]$  for  $0 \leq x \leq y \leq 1$ , that is defined recursively:  $F(\epsilon) = [0, 1]$ ,  $F("0") = [0, 1/2]$ ,  $F("1") = [1/2, 1]$ ,  $F("00") = [0, 1/4]$  and so on. Namely for any given binary string  $b$ ,  $F(b \circ "0")$  and  $F(b \circ "1")$  correspond to the first and second half of  $F(b)$  respectively. WLOG we can assume that the domain of  $\tilde{\mathbf{D}}$  is  $\{0, 1\}^d$ , i.e., the set of binary strings of length  $d$  for some fixed constant  $d$ . Construct  $\mathbf{D}_R$ ,  $\mathbf{D}_S$ , and  $\mathbf{D}_T$  as follows:

$$\mathbf{D}_R := \{(F(a_1 \circ a_2), F(b_1)) \mid (a_1, a_2, b_1) \in \tilde{\mathbf{D}}_{R_{2,1}}\},$$

$$\mathbf{D}_S := \{(F(b_1 \circ b_2), F(c_1 \circ c_2)) \mid (b_1, b_2, c_1, c_2) \in \tilde{\mathbf{D}}_{S_{2,2}}\},$$

$$\mathbf{D}_T := \{(F(a_1), F(c_1)) \mid (a_1, c_1) \in \tilde{\mathbf{D}}_{T_{1,1}}\}.$$

We can show that  $Q_\Delta(\mathbf{D})$  holds if and only if  $\tilde{Q}_3(\tilde{\mathbf{D}})$ .

Moreover  $|\tilde{\mathbf{D}}| = |\mathbf{D}|$ . This basically proves that solving  $Q_\Delta$  is at least as hard as solving  $\tilde{Q}_3$ . The same holds for all queries  $\{\tilde{Q}_1, \dots, \tilde{Q}_8\}$ . In contrast, the forward reduction shows that  $Q_\Delta$  is at most as hard as solving the hardest query among  $\{\tilde{Q}_1, \dots, \tilde{Q}_8\}$ . Together, this implies that  $Q_\Delta$  is exactly as hard as the hardest query among  $\{\tilde{Q}_1, \dots, \tilde{Q}_8\}$ , meaning that our forward reduction is actually tight. See Appendix C for more details.  $\square$

THEOREM 5.2. *Let  $Q$  be any self-join-free IJ query with hypergraph  $\mathcal{H}$ . Let  $\tilde{Q}$  be any EJ query whose hypergraph is in  $\tau(\mathcal{H})$ . For any database  $\tilde{\mathbf{D}}$ , let  $\Omega(T(|\tilde{\mathbf{D}}|))$  be a lower bound on the time complexity for computing  $\tilde{Q}$ , where  $T$  is a function of the size of the database  $\tilde{\mathbf{D}}$ . There cannot be an algorithm  $\mathcal{A}_Q$  that computes  $Q(\mathbf{D})$  in time  $o(T(|\mathbf{D}|))$  (i.e., asymptotically strictly smaller), for any database  $\mathbf{D}$ .*

## 6 IOTA-ACYCLICITY

In this section, we answer the following question: Which IJ queries can be computed in linear time (modulo a polylog factor)? To answer this question, we introduce a new notion of acyclicity, called  $\iota$ -acyclicity, which captures precisely the linear-time computable IJ queries. In other words,  $\iota$ -acyclicity is for IJ queries what  $\alpha$ -acyclicity is for EJ queries.

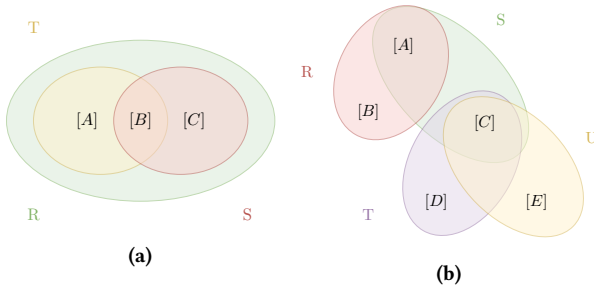
DEFINITION 6.1 (IOTA ACYCLIC HYPERGRAPH). *A hypergraph  $\mathcal{H}$  is  $\iota$ -acyclic if and only if each hypergraph in  $\tau(\mathcal{H})$  is  $\alpha$ -acyclic.*

It is immediate to see why Definition 6.1 defines the hypergraphs of some linear-time computable IJ queries, namely those computed via our reduction. Theorem 6.6 later shows that Definition 6.1 defines in fact all linear-time computable IJ queries.

Since all hypergraphs in  $\tau(\mathcal{H})$  are  $\alpha$ -acyclic, they correspond to EJ queries that can be computed in linear time [29]. Furthermore, the size of  $\tau(\mathcal{H})$  is independent of the input database and only depends on  $\mathcal{H}$ . Definition 6.1 defines  $\iota$ -acyclicity indirectly using our reduction. We next show that this is equivalent to a simple syntactic characterisation of the hypergraph of the given IJ query.

DEFINITION 6.2 (BERGE CYCLE [10]). *A Berge cycle in  $\mathcal{H}$  is a sequence  $(e^1, v^1, e^2, v^2, \dots, e^n, v^n, e^{n+1})$  such that:  $v^1, \dots, v^n$  are distinct vertices in  $\mathcal{V}$ ;  $e^1, \dots, e^n$  are distinct hyperedges in  $\mathcal{E}$  and  $e^{n+1} = e^1$ ;  $n \geq 2$ ; and  $v^i$  is in  $e^i$  and  $e^{i+1}$  for each  $1 \leq i \leq n$ .*

THEOREM 6.3 (IOTA ACYCLICITY CHARACTERISATION). *A hypergraph is  $\iota$ -acyclic if and only if it has no Berge cycle of length strictly greater than two.*



**Figure 4: (a) Hypergraph with Berge cycle of length three and thus not  $\iota$ -acyclic. (b) Hypergraph without a Berge cycle and thus  $\iota$ -acyclic.**

The smallest Berge cycle that makes a hypergraph  $\mathcal{H}$  not  $\iota$ -acyclic has length three. This is a sequence  $(e^1, v^1, e^2, v^2, e^3, v^3, e^4)$ , where  $v^1, v^2, v^3$  are distinct vertices in the hypergraph,  $e^1, e^2, e^3$  are distinct hyperedges in the hypergraph,  $e^4 = e^1$ , and  $v^i \in e^i \cap e^{i+1}$  for  $1 \leq i \leq 3$  (Definition 6.2).

As a corollary of Theorem 6.3,  $\iota$ -acyclicity strictly sits between Berge-acyclicity and  $\gamma$ -acyclicity [7, 10]. A further corollary is that each  $\iota$ -acyclic hypergraph is also  $\alpha$ -acyclic.

**COROLLARY 6.4.** *The class of  $\iota$ -acyclic hypergraphs is a strict superset of the class of Berge-acyclic hypergraphs and it is a strict subset of the class of  $\gamma$ -acyclic hypergraphs.*

**EXAMPLE 6.5.** The hypergraph of the query  $Q = R([A], [B], [C]) \wedge S([A], [B], [C]) \wedge T([A], [B])$  is not  $\iota$ -acyclic since it has the following Berge cycle of length three:  $R - [C] - S - [B] - T - [A] - R$ . It becomes  $\iota$ -acyclic by removing any of its vertices or hyperedges. The hypergraph of the query  $Q = R([A], [B], [C]) \wedge S([A], [B], [C]) \wedge T([A])$  is  $\iota$ -acyclic since it has no Berge cycle of length strictly greater than two. It only has three Berge cycles of length two. Those are the cycles  $R - [A] - S - [B] - R$ ,  $R - [B] - S - [C] - R$ ,  $R - [A] - S - [C] - R$ .

We now turn to the two hypergraphs in Figure 4. The hypergraph in Figure 4a has a Berge cycle of length 3:  $R - [A] - T - [B] - S - [C] - R$ . Applying the translation from intersection joins to equality joins produces  $2! \cdot 3! \cdot 2! = 24$  hypergraphs. We next analyse the reduced versions of the hypergraphs, where we drop the vertices that appear in one hyperedge only. This reduction is justified in our analysis of the  $\iota$ -acyclicity, since such vertices cannot contribute to a Berge cycle. There are only three distinct reduced hypergraphs  $\mathcal{H}_1$ ,  $\mathcal{H}_2$  and  $\mathcal{H}_3$ . We next analyse their widths.

The hypergraph  $\mathcal{H}_1$  has hyperedges  $\tilde{R}(A_1, B_1, C_1)$ ,  $\tilde{S}(B_1, C_1, B_2)$  and  $\tilde{T}(A_1, B_1, B_2)$ . Its fractional hypertree width is 1.5. This is obtained using a hypertree decomposition consisting of the bag  $\{A_1, B_1, C_1, B_2\}$ . This bag covers all hyperedges and has a fractional edge cover number of 1.5. This is obtained by assigning the weights  $[0.5, 0.5, 0.5]$  to the hyperedges.

$\mathcal{H}_2$  has hyperedges  $\tilde{R}(A_1, B_1, C_1, B_2)$ ,  $\tilde{S}(B_1, C_1, B_2)$  and  $\tilde{T}(A_1, B_1)$ . Its fractional hypertree width is 1.0. This is obtained using a hypertree decomposition consisting of the bag  $\{A_1, B_1, C_1, B_2\}$ . This bag covers all hyperedges and has a fractional edge cover number

of 1.0. This is obtained by assigning the weights  $[1.0, 0.0, 0.0]$  to the hyperedges.

$\mathcal{H}_3$  has hyperedges  $\tilde{R}(A_1, B_1, C_1, B_2)$ ,  $\tilde{S}(B_1, C_1)$  and  $\tilde{T}(A_1, B_1, B_2)$ . Its fractional hypertree width is 1.0, witnessed by the same hypertree decomposition as for  $\mathcal{H}_2$ .

The submodular width is the same as the fractional hypertree width for all three hypergraphs. We conclude that the ijw is  $3/2$ , which is the maximum of the above three fractional hypertree widths. Therefore, our approach takes time  $O(N^{3/2} \cdot \text{polylog} N)$  for the IJ query with the hypergraph in Figure 4a.

The hypergraph in Figure 4b has no Berge cycle. Applying the reduction produces  $2! \cdot 1! \cdot 3! \cdot 1! \cdot 1! = 12$  hypergraphs that are all  $\alpha$ -acyclic. Our approach thus takes time  $O(N \cdot \text{polylog} N)$ .  $\square$

IJ queries whose hypergraphs are not  $\iota$ -acyclic cannot be computed in linear time (unless the 3SUM conjecture fails). This is shown by a reduction from the problem of computing the triangle EJ query, which takes super-linear time [3] unless the 3SUM conjecture fails [25]. The 3SUM problem asks, given a set  $S$  of  $n$  numbers, to find distinct  $x, y, z \in S$  such that  $x + y = z$ . The problem can be solved in  $O(n^2)$  time, and it is a long-standing conjecture that this quadratic complexity is essentially the best possible.

**THEOREM 6.6 (IOTA ACYCLOCITY DICHOTOMY).** *Let  $Q$  be any IJ query with hypergraph  $\mathcal{H}$  and let  $D$  be any database.*

*If  $\mathcal{H}$  is  $\iota$ -acyclic, then  $Q$  can be computed in time  $O(|D| \cdot \text{polylog} |D|)$ .*

*If  $\mathcal{H}$  is not  $\iota$ -acyclic, then there is no algorithm that can compute  $Q$  in time  $O(|D|^{4/3-\epsilon})$  for  $\epsilon > 0$ , unless the 3SUM conjecture fails.*

## 7 CONCLUSION AND FUTURE WORK

This paper pinpoints the complexity of Boolean conjunctive queries with intersection joins and characterises syntactically the class of such queries that can be computed in linear time modulo a polylogarithmic factor. Core to our approach is a reduction of the evaluation problem for such queries to Boolean queries with equality joins. This reduction is robust: It also works for non-Boolean queries with both intersection and equality joins.

A natural extension of this work is to refine the acyclicity notion in the presence of both intersection joins and equality joins. This notion necessarily lies between  $\alpha$ -acyclicity and  $\iota$ -acyclicity: It is the former when all joins are equality joins, as in the literature, and it is the latter when all joins are intersection joins, as in this paper. A further type of join that is naturally supported by the development in this paper is the *membership join*: This can be expressed by using a join variable to range over both intervals and points. Our reduction can be optimised to accommodate membership joins, in addition to intersection and equality joins. Characterising the linear-time computable Boolean queries with all three types of joins is an exciting venue of future research.

## ACKNOWLEDGMENTS

This project has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement No 682588. AK gratefully acknowledges support from EPSRC via a CASE grant supported by Ordnance Survey.

## REFERENCES

- [1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.
- [2] Mahmoud Abo Khamis, Ryan R. Curtin, Benjamin Moseley, Hung Q. Ngo, Xuanlong Nguyen, Dan Olteanu, and Maximilian Schleich. 2020. Functional Aggregate Queries with Additive Inequalities. *ACM Trans. Database Syst.* 45, 4, Article 17 (2020), 41 pages.
- [3] Mahmoud Abo Khamis, Hung Q. Ngo, Christopher Ré, and Atri Rudra. 2016. Joins via Geometric Resolutions: Worst Case and Beyond. *ACM Trans. Database Syst.* 41, 4 (2016), 22:1–22:45.
- [4] Mahmoud Abo Khamis, Hung Q. Ngo, and Dan Suciu. 2017. What Do Shannon-type Inequalities, Submodular Width, and Disjunctive Datalog Have to Do with One Another?. In *PODS*. 429–444.
- [5] Lars Arge, Octavian Procopiuc, Sridhar Ramaswamy, Torsten Suel, and Jeffrey Scott Vitter. 1998. Scalable Sweeping-Based Spatial Join. In *VLDB*. 570–581.
- [6] Panagiotis Bours and Nikos Mamoulis. 2017. A Forward Scan based Plane Sweep Algorithm for Parallel Interval Joins. *Proc. VLDB Endow.* 10, 11 (2017), 1346–1357.
- [7] Johann Brault-Baron. 2016. Hypergraph Acyclicity Revisited. *ACM Comput. Surv.* 49, 3 (2016), 54:1–54:26.
- [8] Thomas Brinkhoff, Hans-Peter Kriegel, and Bernhard Seeger. 1993. Efficient Processing of Spatial Joins Using R-Trees. In *SIGMOD*. 237–246.
- [9] Jost Enderle, Matthias Hampel, and Thomas Seidl. 2004. Joining Interval Data in Relational Databases. In *SIGMOD*. 683–694.
- [10] Ronald Fagin. 1983. Degrees of Acyclicity for Hypergraphs and Relational Database Schemes. *J. ACM* 30, 3 (1983), 514–550.
- [11] Dengfeng Gao, Christian S. Jensen, Richard T. Snodgrass, and Michael D. Soo. 2005. Join operations in temporal databases. *VLDB J.* 14, 1 (2005), 2–29.
- [12] Xiao Hu, Ke Yi, and Yufei Tao. 2019. Output-Optimal Massively Parallel Algorithms for Similarity Joins. *ACM Trans. Database Syst.* 44, 2 (2019), 6:1–6:36.
- [13] Edwin H. Jacox and Hanan Samet. 2007. Spatial join techniques. *ACM Trans. Database Syst.* 32, 1 (2007), 7.
- [14] Mahmoud Abo Khamis, George Chichirim, Antonia Kormpa, and Dan Olteanu. 2021. The Complexity of Boolean Conjunctive Queries with Intersection Joins. *CoRR* abs/2106.13342 (2021). <https://arxiv.org/abs/2106.13342>
- [15] Nick Koudas and Kenneth C. Sevcik. 1997. Size Separation Spatial Join. In *SIGMOD*. 324–335.
- [16] Ming-Ling Lo and Chinya V. Ravishankar. 1994. Spatial Joins Using Seeded Trees. In *SIGMOD*. 209–220.
- [17] Ming-Ling Lo and Chinya V. Ravishankar. 1996. Spatial Hash-Joins. In *SIGMOD*. 247–258.
- [18] Nikos Mamoulis. 2011. *Spatial Data Management*. Morgan & Claypool Publishers.
- [19] Nikos Mamoulis and Dimitris Papadias. 2001. Multiway spatial joins. *ACM Trans. Database Syst.* 26, 4 (2001), 424–475.
- [20] Nikos Mamoulis and Dimitris Papadias. 2003. Slot Index Spatial Join. *IEEE Trans. Knowl. Data Eng.* 15, 1 (2003), 211–231.
- [21] Daniel Marx. 2013. Tractable Hypergraph Properties for Constraint Satisfaction and Conjunctive Queries. *J. ACM* 60, 6 (2013), 42:1–42:51.
- [22] Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. 2018. Worst-case Optimal Join Algorithms. *J. ACM* 65, 3 (2018), 16:1–16:40.
- [23] Hung Q. Ngo, Christopher Ré, and Atri Rudra. 2013. Skew strikes back: new developments in the theory of join algorithms. *SIGMOD Rec.* 42, 4 (2013), 5–16.
- [24] Jignesh M. Patel and David J. DeWitt. 1996. Partition Based Spatial-Merge Join. In *SIGMOD*. 259–270.
- [25] Mihai Patrascu. 2010. Towards polynomial lower bounds for dynamic problems. In *STOC*. 603–610.
- [26] Danila Piatov, Sven Helmer, and Anton Dignös. 2016. An interval join optimized for modern hardware. In *ICDE*. 1098–1109.
- [27] Franco P. Preparata and Michael I. Shamos. 2012. *Computational geometry: an introduction*. Springer Science & Business Media.
- [28] Dimitrios Tsitsigkos, Panagiotis Bours, Nikos Mamoulis, and Manolis Terrovitis. 2019. Parallel In-Memory Evaluation of Spatial Joins. In *SIGSPATIAL*. 516–519.
- [29] Mihalis Yannakakis. 1981. Algorithms for Acyclic Database Schemes. In *VLDB*. 82–94.

## A BACKGROUND

DEFINITION A.1 (INDUCED SET [7]). Let  $\mathcal{E}$  be a family of sets. The induced set of  $\mathcal{E}$  on  $S \subseteq \bigcup \mathcal{E}$ , is the set  $\mathcal{E}[S] = \{e \cap S \mid e \in \mathcal{E}\} \setminus \{\emptyset\}$ .

DEFINITION A.2 (MINIMISATION OF A FAMILY OF SETS [7]). Let  $\mathcal{E}$  be a family of sets. The set  $\mathcal{M}(\mathcal{E}) = \{e \in \mathcal{E} \mid \nexists f \in \mathcal{E}, e \subset f\}$  is the minimization of  $\mathcal{E}$ . That is, the subset of hyperedges that are maximal with respect to the inclusion order.

DEFINITION A.3 (CONFORMAL HYPERGRAPH [7]). A hypergraph  $\mathcal{H} = (\mathcal{V}, \mathcal{E})$  is conformal if there is no  $S \subseteq \mathcal{V}$ , with cardinality  $\geq 3$  such that  $\mathcal{M}(\mathcal{E}[S]) = \{S \setminus \{x\} \mid x \in S\}$ .

DEFINITION A.4 (CYCLE-FREE HYPERGRAPH [7]). A hypergraph  $\mathcal{H} = (\mathcal{V}, \mathcal{E})$  is cycle-free if there is no tuple  $(v_1, \dots, v_n)$  with  $n \geq 3$  of pairwise distinct vertices such that  $\mathcal{M}(\mathcal{E}[\{v^i \mid 1 \leq i \leq n\}]) = \{\{v^i, v^{i+1}\} \mid 1 \leq i < n\} \cup \{\{v^n, v^1\}\}$ .

DEFINITION A.5 (ALPHA ACYCLIC HYPERGRAPH [1, 7, 10]). Consider a hypergraph  $\mathcal{H}$ . The following statements are equivalent: (1)  $\mathcal{H}$  is  $\alpha$ -acyclic; (2)  $\mathcal{H}$  is conformal and cycle-free; (3)  $\mathcal{H}$  has a join tree; (4)  $\mathcal{H}$  is GYO reducible to the empty hypergraph.

We denote with  $\text{TD}(\mathcal{H})$  the set of hypertree decompositions of  $\mathcal{H}$ .

DEFINITION A.6 (POLYMATROID [4]). Consider the vertex set  $\mathcal{V}$ . A function  $f : 2^{\mathcal{V}} \rightarrow \mathbb{R}^+$  is a (non-negative) set function on  $\mathcal{V}$ . A set function  $f$  on  $\mathcal{V}$  is: modular if  $f(S) = \sum_{v \in S} f(\{v\})$  for all  $S \subseteq \mathcal{V}$ ; monotone if  $f(X) \leq f(Y)$  whenever  $X \subseteq Y$ ; and submodular if  $f(X \cup Y) + f(X \cap Y) \leq f(X) + f(Y)$  for all  $X, Y \subseteq \mathcal{V}$ .

A monotone, submodular set function  $h : 2^{\mathcal{V}} \rightarrow \mathbb{R}^+$  with  $h(\emptyset) = 0$  is a polymatroid. We use  $\Gamma_{\mathcal{V}}$  to denote the set of all polymatroids  $f : 2^{\mathcal{V}} \rightarrow \mathbb{R}^+$  over the set  $\mathcal{V}$ .

DEFINITION A.7 (EDGE DOMINATED SET FUNCTIONS [4, 21]). Let  $\mathcal{H} = (\mathcal{V}, \mathcal{E})$  be a hypergraph. The set of edge dominated set functions is defined by:  $\text{ED}(\mathcal{H}) := \{h \mid h : 2^{\mathcal{V}} \rightarrow \mathbb{R}^+, h(S) \leq 1, \forall S \in \mathcal{E}\}$ .

DEFINITION A.8 (FRACTIONAL HYPERTREE WIDTH). Consider a hypergraph  $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ . The fractional hypertree width of  $\mathcal{H}$  is defined by:

$$\text{fhtw}(\mathcal{H}) := \min_{(\mathcal{T}, \chi) \in \text{TD}(\mathcal{H})} \max_{t \in V(\mathcal{T})} \rho_{\mathcal{E}}^*(\chi(t)), \quad (1)$$

where  $\rho_{\mathcal{E}}^*(\chi(t))$  denotes the fractional edge cover number of  $\chi(t)$ .

DEFINITION A.9 (SUBMODULAR WIDTH [21]). Consider a hypergraph  $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ . The submodular width of  $\mathcal{H}$  is defined by:

$$\text{subw}(\mathcal{H}) := \max_{h \in \text{ED}(\mathcal{H}) \cap \Gamma_{\mathcal{V}}} \min_{(\mathcal{T}, \chi) \in \text{TD}(\mathcal{H})} \max_{t \in V(\mathcal{T})} h(\chi(t)), \quad (2)$$

where  $\text{ED}(\mathcal{H})$  denotes the set of edge dominated set functions over  $\mathcal{H}$ , and  $\Gamma_{\mathcal{V}}$  denotes the set of polymatroids over  $\mathcal{V}$ .

## B MISSING DETAILS FROM SECTION 4

LEMMA 4.11 Given any EIJ query  $Q$ , interval variable  $[X]$  in  $Q$ , and any database  $\mathbf{D}$ , let the EIJ query  $\tilde{Q}_{[X]}$  and database  $\tilde{\mathbf{D}}_{[X]}$  be constructed as per Definitions 4.9 and 4.7. Then,  $Q(\mathbf{D})$  is true if and only if  $\tilde{Q}_{[X]}(\tilde{\mathbf{D}}_{[X]})$  is true.

PROOF. For simplicity let  $k = |\mathcal{E}_{[X]}|$ .

$\Rightarrow$ : Assume that  $Q(\mathbf{D})$  is true. That means that there exists a set of tuples  $\{t_e \in R(e) \mid e \in \mathcal{E}\}$  that satisfy  $\left(\bigcap_{e \in \mathcal{E}_{[X]}} t_e(X)\right) \neq \emptyset$ , and also they satisfy the rest of the join conditions of the query  $Q$ , i.e. the (intersection or equality) joins on the variables in  $\mathcal{V} \setminus \{[X]\}$ .

By Definition 4.9, for each  $i \in [k]$  there exists a tuple  $\tilde{t}_{\tilde{\sigma}_i} \in \tilde{R}(\tilde{\sigma}_i)$ , where  $\tilde{R}(\tilde{\sigma}_i)$  is a relation over schema  $\tilde{\sigma}_i = (\sigma_i \setminus \{[X]\}) \cup \{X_1, \dots, X_i\}$  in  $\tilde{\mathbf{D}}_{([X], \sigma)}$ , such that  $\tilde{t}_{\tilde{\sigma}_i}(\tilde{\sigma}_i \setminus \{X_1, \dots, X_i\}) = t_{\sigma_i}(\sigma_i \setminus \{[X]\})$  and  $\tilde{t}_{\tilde{\sigma}_i}(X_1) = x_1, \dots, \tilde{t}_{\tilde{\sigma}_i}(X_i) = x_i$ . Furthermore, by Definition 4.9 for each  $e \in \mathcal{E} \setminus \mathcal{E}_{[X]}$  we have  $R(e) \in \tilde{\mathbf{D}}_{([X], \sigma)}$ . For each  $i \in [k]$  we have  $\tilde{t}_{\tilde{\sigma}_i}(X_i) = \tilde{t}_{\tilde{\sigma}_{i+1}}(X_i) = \dots = \tilde{t}_{\tilde{\sigma}_k}(X_i)$ , i.e.

the tuples  $\tilde{t}_{\tilde{\sigma}_1}, \dots, \tilde{t}_{\tilde{\sigma}_k}$  satisfy the equi-join conditions for the variables  $X_1, \dots, X_k$  in the query  $\tilde{Q}_{([X], \sigma)}$ . Furthermore, all the tuples  $\tilde{t}_{\tilde{\sigma}_1}, \dots, \tilde{t}_{\tilde{\sigma}_k}$  and  $\tilde{t}_e$  for each  $e \in \mathcal{E} \setminus \mathcal{E}_{[X]}$  satisfy the rest of the join conditions of query  $\tilde{Q}_{([X], \sigma)}$  i.e. the (intersection or equality) joins on variables in  $\mathcal{V} \setminus \{[X]\}$ . Hence,  $\tilde{Q}_{([X], \sigma)}(\tilde{\mathbf{D}}_{([X], \sigma)})$  is true. Therefore,  $\tilde{Q}_{[X]}(\tilde{\mathbf{D}}_{[X]})$  is true.

$\Leftarrow$ : Assume that  $\tilde{Q}_{[X]}(\tilde{\mathbf{D}}_{[X]})$  is true. That means that there exists a permutation  $\sigma \in \pi(\mathcal{E}_{[X]})$  and there exist tuples  $\tilde{t}_{\tilde{\sigma}_i} \in \tilde{R}(\tilde{\sigma}_i), \dots, \tilde{t}_{\tilde{\sigma}_k} \in \tilde{R}(\tilde{\sigma}_k)$  and  $\tilde{t}_e \in R(e)$  for each  $e \in \mathcal{E} \setminus \mathcal{E}_{[X]}$  such that for each  $i \in [k]$  we have  $\tilde{t}_{\tilde{\sigma}_i}(X_i) = \tilde{t}_{\tilde{\sigma}_{i+1}}(X_i) = \dots = \tilde{t}_{\tilde{\sigma}_k}(X_i)$  and also the tuples satisfy the rest of the join conditions of the query  $\tilde{Q}_{[X]}$ , i.e. the (intersection or equality) joins on variables in  $\mathcal{V} \setminus \{[X]\}$ .

By Definition 4.9, for each  $i \in [k]$  there exists a tuple  $t_{\sigma_i} \in R(\sigma_i)$ , where  $R(\sigma_i)$  is a relation over schema  $\sigma_i = (\tilde{\sigma}_i \cup \{[X]\}) \setminus \{X_1, \dots, X_k\}$  in  $\mathbf{D}$ , such that  $t_{\sigma_i}(\sigma_i \setminus \{[X]\}) = \tilde{t}_{\tilde{\sigma}_i}(\tilde{\sigma}_i \setminus \{X_1, \dots, X_i\})$  and  $\tilde{t}_{\tilde{\sigma}_i}(X_1) \circ \dots \circ \tilde{t}_{\tilde{\sigma}_i}(X_i) \in \text{CP}_I(t_{\sigma_i}(X))$  for each  $i \in [k-1]$  and  $\tilde{t}_{\tilde{\sigma}_i}(X_1) \circ \dots \circ \tilde{t}_{\tilde{\sigma}_i}(X_i) = \text{leaf}(t_{\sigma_i}(X))$  for  $i = k$ . Furthermore, by Definition 4.9 we have  $R(e) \in \mathbf{D}$  for each  $e \in \mathcal{E}$ . By Lemma 4.4, the predicate  $(\bigcap_{i \in [k]} t_{\sigma_i}(X)) \neq \emptyset$  is true. Therefore, the tuples  $t_{\sigma_1}, \dots, t_{\sigma_k}$  satisfy the intersection join condition on variable  $[X]$  in query  $Q$ . Furthermore, all the tuples  $t_{\sigma_1}, \dots, t_{\sigma_k}$  and  $t_e$  for each  $e \in \mathcal{E} \setminus \mathcal{E}_{[X]}$  satisfy the rest of the join conditions of  $Q$ , i.e. the (intersection or equality) joins on variables in  $\mathcal{V} \setminus \{[X]\}$ . Hence  $Q(\mathbf{D})$  is true.  $\square$

## C MISSING DETAILS FROM SECTION 5

**THEOREM 5.2** Let  $Q$  be any self-join-free IJ query with hypergraph  $\mathcal{H}$ . Let  $\tilde{Q}$  be any EJ query whose hypergraph is in  $\tau(\mathcal{H})$ . For any database  $\tilde{\mathbf{D}}$ , let  $\Omega(T(|\tilde{\mathbf{D}}|))$  be a lower bound on the time complexity for computing  $\tilde{Q}$ , where  $T$  is a function of the size of the database  $\tilde{\mathbf{D}}$ . There cannot be an algorithm  $\mathcal{A}_Q$  that computes  $Q(\mathbf{D})$  in time  $o(T(|\mathbf{D}|))$  (i.e., asymptotically strictly smaller), for any database  $\mathbf{D}$ .

**PROOF.** Suppose, for a contradiction, that there is such an algorithm  $\mathcal{A}_Q$ . We will show that we can construct an algorithm  $\mathcal{A}_{\tilde{Q}}$  based on  $\mathcal{A}_Q$  that can solve  $\tilde{Q}(\tilde{\mathbf{D}})$  in time complexity  $o(T(|\tilde{\mathbf{D}}|))$  (i.e., asymptotically strictly smaller), for any input database  $\tilde{\mathbf{D}}$  of  $\tilde{Q}$ .

Let  $\tilde{\mathbf{D}}$  be any input database for EJ query  $\tilde{Q}$ . We will base our construction on the structure of the segment tree. Let  $\text{brep}(x)$  be the binary representation of the natural number  $x$ . WLOG we can assume that each value in  $\tilde{\mathbf{D}}$  is a binary string of length exactly  $b$  for some constant  $b$ . Let  $n = |\mathcal{E}| = |\tilde{\mathcal{E}}|$ .

Consider a slightly modified version of a perfect segment tree with  $2^{n \cdot b}$  leaves (so with height equal to  $n \cdot b$ ) where, for each node  $u$ , we have  $\text{seg}(u) := [x, y]$ , where  $x$  and  $y$  are natural numbers such that  $\text{brep}(x) := "1" \circ u \circ "0"^\ell$  and  $\text{brep}(y) := "1" \circ u \circ "1"^\ell$ , where  $\ell := n \cdot b - |u|$  and  $"0"^\ell$  represents the string "0" repeated  $\ell$  times (and the same for  $"1"^\ell$ ). All the properties of a segment tree that are relevant for this proof hold for this version too.

Similar to the one-step forward reduction from Section 4.2, we define a one-step backward reduction and then apply it repeatedly.

**DEFINITION C.1 (ONE-STEP BACKWARD DATABASE TRANSFORMATION).** Given an EJ  $Q$ , let  $[X]$  be an interval variable of  $Q$  and

let  $\sigma \in \pi(\mathcal{E}_{[X]})$ . Let  $\tilde{Q}_{([X], \sigma)}$  be the EJ resulting from the one-step query rewriting from Definition 4.7. Let  $\tilde{\mathbf{D}}_{([X], \sigma)}$  be an arbitrary database instance over the schema of  $\tilde{Q}_{([X], \sigma)}$ . We construct a database instance  $\mathbf{D}$  over the schema of  $Q$  as follows. For each tuple  $\tilde{t} \in \tilde{R}(\tilde{\sigma}_i)$ , we construct a tuple  $t \in R(\sigma_i)$  such that:

- $t(\sigma_i \setminus \{[X]\}) = \tilde{t}(\tilde{\sigma}_i \setminus \{X_1, \dots, X_i\})$
- $t([X]) = \text{seg}(\tilde{t}(X_1) \circ \dots \circ \tilde{t}(X_i))$

All relations in  $\tilde{\mathbf{D}}_{([X], \sigma)}$  other than  $\{R(\sigma_i) \mid i \in [k]\}$  where  $k := |\mathcal{E}_{[X]}|$  are copied directly to  $\mathbf{D}$ .

Note that by the above definition, we have a bijection between tuples  $\tilde{t}$  from  $\tilde{\mathbf{D}}$  and tuples  $t$  from  $\mathbf{D}$ . Let  $g$  be a function that maps a tuple  $\tilde{t}$  in  $\tilde{\mathbf{D}}$  to the corresponding tuple  $t$  from  $\mathbf{D}$ . Because of this bijection, we also have  $|\mathbf{D}| = |\tilde{\mathbf{D}}_{([X], \sigma)}|$ .

**CLAIM C.2.** Given  $Q$ ,  $\tilde{Q}_{([X], \sigma)}$  and  $\tilde{\mathbf{D}}_{([X], \sigma)}$  from Definition C.1,  $Q(\mathbf{D})$  holds if and only if  $\tilde{Q}_{([X], \sigma)}(\tilde{\mathbf{D}}_{([X], \sigma)})$  holds.

**PROOF OF CLAIM C.2.** I)  $\Leftarrow$ : Assume that  $\tilde{Q}_{([X], \sigma)}(\tilde{\mathbf{D}}_{([X], \sigma)})$ . Let  $\tilde{\mathcal{H}}_{([X], \sigma)}$  be the corresponding hypergraph whose hyperedges are  $\tilde{\mathcal{E}}_{([X], \sigma)}$ . This means that each relation  $\tilde{R}(\tilde{e})$  in  $\tilde{\mathbf{D}}_{([X], \sigma)}$  contains a tuple  $\tilde{t}_{\tilde{e}}$  such that the tuples  $(\tilde{t}_{\tilde{e}})_{\tilde{e} \in \tilde{\mathcal{E}}_{([X], \sigma)}}$  satisfy  $\tilde{Q}_{([X], \sigma)}$ . Let  $k := |\mathcal{E}_{[X]}|$ . There must exist a tuple  $(x_1, \dots, x_k)$  where for each  $i \in [k]$ , we have  $\tilde{t}_{\tilde{\sigma}_i}(X_1) = x_1, \dots, \tilde{t}_{\tilde{\sigma}_i}(X_i) = x_i$ . Therefore the binary strings  $\{\tilde{t}_{\tilde{\sigma}_i}(X_1) \circ \dots \circ \tilde{t}_{\tilde{\sigma}_i}(X_i) \mid i \in [k]\}$  are a prefix of one another. By Property (1) from 3.2, this means that the line segments  $\{\text{seg}(\tilde{t}_{\tilde{\sigma}_i}(X_1) \circ \dots \circ \tilde{t}_{\tilde{\sigma}_i}(X_i)) \mid i \in [k]\}$  intersect. Therefore, the tuples  $(t_e := g(\tilde{t}_{\tilde{e}}))_{\tilde{e} \in \tilde{\mathcal{E}}_{([X], \sigma)}}$  satisfy  $Q$ .

II)  $\Rightarrow$ : Assume that  $Q(\mathbf{D})$  is true. This means that each relation  $R_e$  contains a tuple  $t_e$  such that the tuples  $(t_e)_{e \in \mathcal{E}}$  satisfy  $Q$ . In particular, the intervals  $\{t_{\sigma_i}([X]) \mid i \in [k]\}$  intersect. By Definition C.1, the tuples  $(\tilde{t}_{\tilde{\sigma}_i} := g^{-1}(t_{\sigma_i})) \in \tilde{R}(\tilde{\sigma}_i)$  satisfy  $t_{\sigma_i}([X]) = \text{seg}(\tilde{t}_{\tilde{\sigma}_i}(X_1) \circ \dots \circ \tilde{t}_{\tilde{\sigma}_i}(X_i))$  for  $i \in [k]$ . By Property (1) from 3.2, the binary strings  $\{\tilde{t}_{\tilde{\sigma}_i}(X_1) \circ \dots \circ \tilde{t}_{\tilde{\sigma}_i}(X_i) \mid i \in [k]\}$  are a prefix of one another. Moreover for each  $i \in [k]$ , the binary strings  $\tilde{t}_{\tilde{\sigma}_i}(X_1), \dots, \tilde{t}_{\tilde{\sigma}_i}(X_i)$  have the same length which is  $b$  (by our assumption about  $\tilde{\mathbf{D}}$ ). Hence, for each  $j \in [k]$ ,  $\tilde{t}_{\tilde{\sigma}_j}(X_j) = \tilde{t}_{\tilde{\sigma}_{j+1}}(X_j) = \dots = \tilde{t}_{\tilde{\sigma}_k}(X_j)$ . Therefore, the tuples  $(\tilde{t}_{\tilde{e}} := g^{-1}(t_e))_{e \in \mathcal{E}}$  satisfy  $\tilde{Q}_{([X], \sigma)}$ .  $\square$

Finally we use the above claim to finish the proof of Theorem 5.2. By repeatedly applying the above reduction on the EJ query  $\tilde{Q}$  and its database instance  $\tilde{\mathbf{D}}$ , we construct a database instance  $\mathbf{D}$  such that  $Q(\mathbf{D})$  holds if and only if  $\tilde{Q}(\tilde{\mathbf{D}})$  holds. Moreover by Definition C.1, we have  $|\mathbf{D}| = |\tilde{\mathbf{D}}|$ . Now we use the algorithm  $\mathcal{A}_Q$  to answer  $Q(\mathbf{D})$  in time  $o(T(|\mathbf{D}|))$ , thus resulting in an algorithm  $\mathcal{A}_{\tilde{Q}}$  that can answer  $\tilde{Q}(\tilde{\mathbf{D}})$  in time  $o(T(|\tilde{\mathbf{D}}|))$ . But this is a contradiction since  $\tilde{Q}(\tilde{\mathbf{D}})$  has a lower bound of  $\Omega(T(|\tilde{\mathbf{D}}|))$ .  $\square$

## D MISSING DETAILS FROM SECTION 6

Consider an IJ  $Q$  as per Definition 3.3, and let  $\mathcal{H} = (\mathcal{V}, \mathcal{E})$  be the hypergraph of  $Q$ . We denote the vertices of  $\mathcal{H}$  by letters (e.g.  $u$ ) and the vertices of any  $\tilde{\mathcal{H}} = (\tilde{\mathcal{V}}, \tilde{\mathcal{E}}) \in \tau(\mathcal{H})$  by letters with tilde (e.g.  $\tilde{u}$ ). For each vertex  $u \in \mathcal{V}$ , let  $n_u = |\mathcal{E}_u|$  be the number of hyperedges that contain  $u$ . Each vertex  $u$  that occurs in  $n_u$  hyperedges in  $\mathcal{H}$  corresponds to  $n_u$  vertices in  $\tilde{\mathcal{H}}$  denoted by  $\tilde{u}_1, \dots, \tilde{u}_{n_u}$ .

**DEFINITION D.1.** Consider a hypergraph  $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ . Let  $\tilde{\mathcal{H}} = (\tilde{\mathcal{V}}, \tilde{\mathcal{E}})$  be any member of  $\tau(\mathcal{H})$ . (1) Let  $v_{\tilde{\mathcal{H}}, \mathcal{H}} : \tilde{\mathcal{V}} \rightarrow \mathcal{V}$  be the surjective function that maps each vertex  $\tilde{u} \in \tilde{\mathcal{V}}$  to the corresponding vertex  $u \in \mathcal{V}$ . (2) Let  $e_{\tilde{\mathcal{H}}, \mathcal{H}} : \tilde{\mathcal{E}} \rightarrow \mathcal{E}$  be the bijective function that maps each hyperedge  $\tilde{e} \in \tilde{\mathcal{E}}$  to the corresponding hyperedge  $e = \{u \mid \tilde{u} \in \tilde{e}\} \in \mathcal{E}$ . Whenever it is clear from the context, we omit the subscript from the names of the functions.

**PROPERTY D.2 (PROPERTIES OF IJ -TO-EJ REDUCTION).** Consider a hypergraph  $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ . Let  $\tilde{\mathcal{H}} = (\tilde{\mathcal{V}}, \tilde{\mathcal{E}})$  be any member of  $\tau(\mathcal{H})$ .

- (1) For each hyperedge  $\tilde{e} \in \tilde{\mathcal{E}}$  and each vertex  $\tilde{u} \in \tilde{\mathcal{V}}$ , if  $\tilde{u} \in \tilde{e}$  in  $\tilde{\mathcal{H}}$  then  $v(\tilde{u}) \in e(\tilde{e})$  in  $\mathcal{H}$ .
- (2) For each hyperedge  $\tilde{e} \in \tilde{\mathcal{E}}$  and each vertex  $u \in \mathcal{V}$ ,  $\tilde{u}_1 \in \tilde{e}$  in  $\tilde{\mathcal{H}}$  if and only if  $u \in e(\tilde{e})$  in  $\mathcal{H}$ .
- (3) For any two vertices  $\tilde{u}_i, \tilde{u}_j \in \tilde{\mathcal{V}}$  with  $i < j$  and  $\tilde{e} \in \tilde{\mathcal{E}}$ , we have that if  $\tilde{u}_j \in \tilde{e}$  then  $\tilde{u}_i \in \tilde{e}$ .

**LEMMA D.3.** If  $\tilde{\mathcal{H}}$  has a Berge cycle  $(\tilde{e}^1, \tilde{v}^1, \tilde{e}^2, \tilde{v}^2, \dots, \tilde{e}^k, \tilde{v}^k, \tilde{e}^{k+1} = \tilde{e}^1)$  of length  $k$  such that  $v(\tilde{v}^1), \dots, v(\tilde{v}^k)$  are pairwise distinct vertices from  $\mathcal{V}$ , then  $\mathcal{H}$  also has a Berge cycle of length  $k$ .

**THEOREM 6.3** A hypergraph  $\mathcal{H} = (\mathcal{V}, \mathcal{E})$  is  $\iota$ -acyclic if and only if  $\mathcal{H}$  has no Berge cycle of length strictly greater than two.

**PROOF.**  $\Rightarrow$ : Assume for a contradiction that  $\mathcal{H}$  has a Berge cycle of length strictly greater than two, or equivalently at least three. Hence, there exist a cyclic sequence  $(e^1, v^1, e^2, v^2, \dots, e^k, v^k, e^{k+1} = e^1)$  such that  $k \geq 3$ ,  $v^1, \dots, v^k$  are pairwise distinct vertices from  $\mathcal{V}$ ,  $e^1, \dots, e^k$  are pairwise distinct hyperedges from  $\mathcal{E}$ , and for each  $1 \leq i \leq k$ , we have  $v^i \in e^i$  and  $v^i \in e^{i+1}$ . By our construction in Algorithm 1, there exists a hypergraph  $\tilde{\mathcal{H}} = (\tilde{\mathcal{V}}, \tilde{\mathcal{E}}) \in \tau(\mathcal{H})$  such that for each  $1 \leq i \leq k$  we have:  $\{\tilde{v}_1^i, \dots, \tilde{v}_{n_{v^i}-1}^i\} \subseteq \epsilon^{-1}(e^i)$  and  $\tilde{v}_{n_{v^i}}^i \notin \epsilon^{-1}(e^i)$  and  $\{\tilde{v}_1^i, \dots, \tilde{v}_{n_{v^i}}^i\} \subseteq \epsilon^{-1}(e^{i+1})$ . Since  $k \geq 3$ ,  $\tilde{\mathcal{H}}$  has the following three properties:

- (1) For each  $1 \leq i \leq k$  the vertex  $\tilde{v}_{n_{v^i}-1}^i$  belongs to precisely two hyperedges from  $\tilde{\mathcal{E}}$  (the hyperedges  $\epsilon^{-1}(e^i)$  and  $\epsilon^{-1}(e^{i+1})$ );
- (2) For each  $1 \leq i, j \leq k$  with  $|i - j| \leq 2$ , the hyperedge  $\epsilon^{-1}(e^i)$  cannot be contained in hyperedge  $\epsilon^{-1}(e^j)$  because, by (1) the vertex  $\tilde{v}_{n_{v^i}-1}^i \in \epsilon^{-1}(e^i)$  and  $\tilde{v}_{n_{v^i}-1}^i \notin \epsilon^{-1}(e^j)$ ;
- (3) For each  $1 \leq i \leq k$ , the hyperedges  $\epsilon^{-1}(e^i)$  and  $\epsilon^{-1}(e^{i+1})$  cannot be subset of each other because by (1) we have:
  - $\tilde{v}_{n_{v^i}-1}^{i-1}$  belongs to  $\epsilon^{-1}(e^i)$  but cannot belong to  $\epsilon^{-1}(e^{i+1})$ ;
  - $\tilde{v}_{n_{v^{i+1}}-1}^{i+1}$  belongs to  $\epsilon^{-1}(e^{i+1})$  but cannot belong to  $\epsilon^{-1}(e^i)$ .

Let  $\tilde{\mathcal{V}}' = \{\tilde{v}_{n_{v^i}-1}^i \mid 1 \leq i \leq k\}$  and  $\tilde{\mathcal{E}}' = \{\epsilon^{-1}(e^i) \mid 1 \leq i \leq k\}$ . Note that  $\tilde{\mathcal{V}}' \subseteq \tilde{\mathcal{V}}$  and  $\tilde{\mathcal{E}}' \subseteq \tilde{\mathcal{E}}$ . Therefore, no matter what other steps are taken during the runtime of the GYO reduction on  $\tilde{\mathcal{H}}$ , by (1), no vertex from  $\tilde{\mathcal{V}}'$  will become candidate for removal, and by (2) and (3), no hyperedge from  $\tilde{\mathcal{E}}'$  will become candidate for removal. Hence,  $\tilde{\mathcal{H}}$  cannot be GYO reducible to the empty hypergraph. In other words, the hypergraph  $\tilde{\mathcal{H}}$  includes the cycle  $\{\{\tilde{v}_{n_{v^i}-1}^i, \tilde{v}_{n_{v^{i+1}}-1}^{i+1}\} \mid 1 \leq i < k\} \cup \{\{\tilde{v}_{n_{v^k}-1}^k, \tilde{v}_{n_{v^1}-1}^1\}\}$ , where, by the above property (1), the edge  $\{\tilde{v}_{n_{v^i}-1}^i, \tilde{v}_{n_{v^{i+1}}-1}^{i+1}\}$  for each  $1 \leq i < k$  is included in precisely one hyperedge from  $\tilde{\mathcal{E}}$ , that is  $\epsilon^{-1}(e^{i+1})$ , and  $\{\tilde{v}_{n_{v^k}-1}^k, \tilde{v}_{n_{v^1}-1}^1\}$  is included in precisely one hyperedge from  $\tilde{\mathcal{E}}$ ,

that is  $\epsilon^{-1}(e^1) (= \epsilon^{-1}(e^{k+1}))$ . Thus,  $\tilde{\mathcal{H}}$  is not  $\alpha$ -acyclic. Therefore, by Definition 6.1 of  $\iota$ -acyclicity,  $\mathcal{H}$  is not  $\iota$ -acyclic. Contradiction.

$\Leftarrow$ : Assume for a contradiction that  $\mathcal{H}$  is not  $\iota$ -acyclic. Hence, by Definition 6.1, there exists  $\tilde{\mathcal{H}} = (\tilde{\mathcal{V}}, \tilde{\mathcal{E}}) \in \tau(\mathcal{H})$  that is not  $\alpha$ -acyclic. Therefore, by Definition A.5,  $\tilde{\mathcal{H}}$  is not conformal or not cycle-free. Next, we contradict each of the two cases.

**Case 1.** The hypergraph  $\tilde{\mathcal{H}}$  is not conformal. Therefore, there exists a subset  $S \subseteq \tilde{\mathcal{V}}$  with  $|S| \geq 3$  such that  $\mathcal{M}(\tilde{\mathcal{E}}[S]) = \{S \setminus \{x\} \mid x \in S\}$ . According to Definition A.1 of an *induced set*, we have  $\tilde{\mathcal{E}}[S] = \{e \cap S \mid e \in \tilde{\mathcal{E}}\} \setminus \emptyset$ . According to Definition A.2 of the *minimisation of a family of sets*, we have  $\mathcal{M}(\tilde{\mathcal{E}}[S]) = \{e \in \tilde{\mathcal{E}}[S] \mid \nexists f \in \tilde{\mathcal{E}}[S], e \subset f\}$ . Let  $\tilde{x}, \tilde{y}, \tilde{z} \in S$  be distinct vertices from  $S$ . Let  $\tilde{e}_{\tilde{x}} = S \setminus \{\tilde{x}\}$ ,  $\tilde{e}_{\tilde{y}} = S \setminus \{\tilde{y}\}$ , and  $\tilde{e}_{\tilde{z}} = S \setminus \{\tilde{z}\}$  be distinct hyperedges from  $\mathcal{M}(\tilde{\mathcal{E}}[S])$ . Since  $S \subseteq \tilde{\mathcal{V}}$ , we also have that  $\tilde{x}, \tilde{y}, \tilde{z} \in \tilde{\mathcal{V}}$ .

We now need the following claim at this point in the proof; its own proof is given at the end of this section.

**CLAIM D.4.** The vertices  $v(\tilde{x})$ ,  $v(\tilde{y})$ , and  $v(\tilde{z})$  are pairwise distinct.

By Definition A.2 of the minimization of a family of sets, we have  $\mathcal{M}(\tilde{\mathcal{E}}[S]) \subseteq \tilde{\mathcal{E}}[S]$ , hence,  $e_{\tilde{x}}, e_{\tilde{y}}$ , and  $e_{\tilde{z}}$  belong also to  $\tilde{\mathcal{E}}[S]$ . By Definition A.1 of the induced set, this means that there exist three distinct hyperedges  $\tilde{c}_{\tilde{x}}, \tilde{c}_{\tilde{y}}, \tilde{c}_{\tilde{z}} \in \tilde{\mathcal{E}}$  such that  $\tilde{e}_{\tilde{x}} \subseteq \tilde{c}_{\tilde{x}}$ ,  $e_{\tilde{y}} \subseteq \tilde{c}_{\tilde{y}}$ , and  $e_{\tilde{z}} \subseteq \tilde{c}_{\tilde{z}}$ . Therefore, the sequence  $(\tilde{c}_{\tilde{x}}, \tilde{z}, \tilde{c}_{\tilde{y}}, \tilde{x}, \tilde{c}_{\tilde{z}}, \tilde{y}, \tilde{e}_{\tilde{x}})$  is a Berge cycle of length 3 in  $\tilde{\mathcal{H}}$  where, by Claim D.4,  $v(\tilde{x}), v(\tilde{y}), v(\tilde{z})$  are pairwise distinct vertices of  $\mathcal{H}$ . Therefore, by Lemma D.3,  $\mathcal{H}$  has also a Berge cycle of length 3. This statement contradicts the initial assumption that  $\mathcal{H}$  is  $\iota$ -acyclic.

**Case 2.** The hypergraph  $\tilde{\mathcal{H}}$  is non-cycle-free. Hence, there exist  $S = \{\tilde{v}^1, \dots, \tilde{v}^k\} \subseteq \tilde{\mathcal{V}}$  where  $k \geq 3$  of pairwise distinct vertices such that  $\mathcal{M}(\tilde{\mathcal{E}}[S]) = \{\{\tilde{v}^i, \tilde{v}^{i+1}\} \mid 1 \leq i < k\} \cup \{\{\tilde{v}^k, \tilde{v}^1\}\}$ . Let  $\tilde{e}^{i+1} := \{\tilde{v}^i, \tilde{v}^{i+1}\}$  for each  $1 \leq i < k$ , and  $\tilde{e}^1 := \{\tilde{v}^k, \tilde{v}^1\}$ .

We now make the following claim; its own proof is given at the end of this section.

**CLAIM D.5.** The vertices  $v(\tilde{v}^1), \dots, v(\tilde{v}^k)$  are pairwise distinct.

By Definition A.2 of the minimization of a family of sets we have  $\mathcal{M}(\tilde{\mathcal{E}}[S]) \subseteq \tilde{\mathcal{E}}[S]$ , this means  $\tilde{e}^i \in \tilde{\mathcal{E}}[S]$  for each  $1 \leq i \leq k$ . By Definition A.1 of the induced set, there exist  $k$  pairwise distinct hyperedges  $\tilde{c}^1, \dots, \tilde{c}^k \in \tilde{\mathcal{E}}$  such that  $\tilde{e}^i \subseteq \tilde{c}^i$  for each  $1 \leq i \leq k$ . Since  $\tilde{c}^1, \dots, \tilde{c}^k$  are distinct hyperedges in  $\tilde{\mathcal{E}}$  and  $\tilde{v}^1, \dots, \tilde{v}^k$  are distinct vertices in  $\tilde{\mathcal{V}}$ , the sequence  $(\tilde{c}^1, \tilde{v}^1, \tilde{c}^2, \tilde{v}^2, \dots, \tilde{c}^k, \tilde{v}^k, \tilde{c}^1)$  is a Berge cycle of length  $k \geq 3$  in  $\tilde{\mathcal{H}}$ . Moreover, by Claim D.5,  $v(\tilde{v}^1), \dots, v(\tilde{v}^k)$  are pairwise distinct vertices of  $\mathcal{H}$ . Therefore, by Lemma D.3,  $\mathcal{H}$  has a Berge-cycle of length  $k \geq 3$ . This statement contradicts the initial assumption that  $\mathcal{H}$  is  $\iota$ -acyclic.

Finally, we give the proofs of Claims D.4 and D.5.

**PROOF OF CLAIM D.4.** Assume for contradiction that there are two distinct vertices  $\tilde{u}, \tilde{v} \in \{\tilde{x}, \tilde{y}, \tilde{z}\}$  such that  $v(\tilde{u}) = v(\tilde{v})$ . By Property D.2 (3), this means that any hyperedge  $\tilde{e} \in \tilde{\mathcal{E}}$  that contains  $\tilde{u}$  contains also vertex  $\tilde{v}$  (or vice versa). Note that, since  $\mathcal{M}(\tilde{\mathcal{E}}[S]) \subseteq \tilde{\mathcal{E}}[S]$  and since  $\tilde{u}, \tilde{v} \in S$ , the property of the previous statement holds also for the hyperedges of  $\mathcal{M}(\tilde{\mathcal{E}}[S])$ . This violates the condition that  $\mathcal{M}(\tilde{\mathcal{E}}[S]) = \{S \setminus \{x\} \mid x \in S\}$  since in this case the hyperedge  $S \setminus \{\tilde{v}\}$  (which contains vertex  $\tilde{u}$ ) would actually need to include

vertex  $\tilde{v}$  as well. The reverse case is analogous due to symmetry. Contradiction.

PROOF OF CLAIM D.5. The proof is similar to the proof of Claim D.4  $\square$

## E SOLVING THE TRIANGLE IJ USING FAQ-AI

Consider the triangle intersection join query  $Q_\Delta$  from Section 1.1. While we showed in Section 1.1 that our approach solves this query in time  $O(N^{3/2} \log^3 N)$ , we show here that the FAQ-AI approach [2] needs time  $O(N^2 \log^3 N)$  for this query.

To apply the FAQ-AI approach to query  $Q_\Delta$  above, we have to express it as a query with inequality joins. In particular, each interval variable (say  $[A]$ ) is going to be replaced by two scalar variables ( $A.l$  and  $A.r$ ) representing the beginning and end of  $[A]$ , i.e.  $[A] = [A.l, A.r]$ . Let  $A.l^{(R)}$  and  $A.r^{(R)}$  be the beginning and end of interval  $[A]$  in relation  $R$ . Similarly, we define  $A.l^{(T)}$  and  $A.r^{(T)}$ . For the intervals  $[A.l^{(R)}, A.r^{(R)}]$  and  $[A.l^{(T)}, A.r^{(T)}]$  to overlap, the following condition must hold:

$$(A.l^{(T)} \leq A.l^{(R)} \leq A.r^{(T)}) \vee (A.l^{(R)} \leq A.l^{(T)} \leq A.r^{(R)}) \quad (3)$$

For each variable  $X \in \{A, B, C\}$ , let  $F(X)$  denote the set of relations containing  $X$ , i.e.  $F(A) := \{T, R\}$ ,  $F(B) := \{R, S\}$ , and  $F(C) := \{S, T\}$ . Note that (3) can be written equivalently as:

$$\bigvee_{\substack{V \in F(A) \\ W \in F(A) - \{V\}}} A.l^{(W)} \leq A.l^{(V)} \leq A.r^{(W)} \quad (4)$$

The same applies to the other two interval variables  $[B]$  and  $[C]$ .

Let:

$$\begin{aligned} \mathcal{R} &:= R(A.l^{(R)}, A.r^{(R)}, B.l^{(R)}, B.r^{(R)}), \\ \mathcal{S} &:= S(B.l^{(S)}, B.r^{(S)}, C.l^{(S)}, C.r^{(S)}), \\ \mathcal{T} &:= T(A.l^{(T)}, A.r^{(T)}, C.l^{(T)}, C.r^{(T)}). \end{aligned}$$

and

$$\begin{aligned} \mathcal{A} &:= (A.l^{(W_A)} \leq A.l^{(V_A)} \leq A.r^{(W_A)}), \\ \mathcal{B} &:= (B.l^{(W_B)} \leq B.l^{(V_B)} \leq B.r^{(W_B)}), \\ \mathcal{C} &:= (C.l^{(W_C)} \leq C.l^{(V_C)} \leq C.r^{(W_C)}). \end{aligned}$$

After distributing disjunctions over conjunctions,  $Q_\Delta$  can be written as follows:

$$Q_\Delta = \bigvee_{\substack{(V_A, V_B, V_C) \in F(A) \times F(B) \times F(C) \\ W_A \in F(A) - \{V_A\} \\ W_B \in F(B) - \{V_B\} \\ W_C \in F(C) - \{V_C\}}} \mathcal{R} \wedge \mathcal{S} \wedge \mathcal{T} \wedge \mathcal{A} \wedge \mathcal{B} \wedge \mathcal{C}. \quad (5)$$

For each  $(V_A, V_B, V_C) \in F(A) \times F(B) \times F(C)$ , the inner conjunction in (5) is an FAQ-AI query [2]. While solving each such query, it is possible to relax the definition of tree decompositions thus extending the set of valid tree decompositions and potentially reducing the fractional hypertree and submodular widths, ultimately resulting in the *relaxed* versions of these widths  $\text{fhtw}_\ell$  and  $\text{subw}_\ell$  respectively [2]. In particular, in a relaxed tree decomposition, we no longer require each inequality to have its variables contained in

one bag of the tree. Instead, it suffices to have its variables contained in two adjacent bags in the tree.

Fix an arbitrary  $(V_A, V_B, V_C) \in F(A) \times F(B) \times F(C)$  and let  $\bar{Q}$  be the resulting FAQ-AI query corresponding to the inner conjunction in (5). Note that for every pair of the relations  $R, S$  and  $T$ , the query  $\bar{Q}$  contains at least one inequality between two variables from that pair. Hence if we distribute the relations  $R, S$  and  $T$  among three or more bags, there will be an inequality between two non-adjacent bags thus violating the condition for a relaxed tree decomposition. Therefore, every relaxed tree decomposition of  $\bar{Q}$  must have at most two bags where each one the relations  $R, S$  and  $T$  falls within one bag. Consequently, there will be one bag with (at least) two relations. Noting that the variables of relations  $R, S$  and  $T$  are pairwise disjoint, this implies that  $\text{fhtw}_\ell(\bar{Q}) \geq 2$ . To minimize  $\text{fhtw}_\ell(\bar{Q})$ , an optimal tree decomposition would have two bags with two relations in one bag and the third relation in the other, resulting in  $\text{fhtw}_\ell(\bar{Q}) = 2$ .

The relaxed submodular width  $\text{subw}_\ell(\bar{Q})$  is not any better in this case. Consider the following function  $\bar{h} : 2^{\text{vars}(\bar{Q})} \rightarrow \mathbb{R}^+$ :

$$\bar{h}(X) := \frac{|X|}{4}, \quad \forall X \subseteq \text{vars}(\bar{Q}). \quad (6)$$

Recall notation from Section A and [2, 21]. The above  $\bar{h}$  is a modular function hence it is submodular. Since it is also monotone,  $\bar{h}$  is a polymatroid, i.e.  $\bar{h} \in \Gamma_{\text{vars}(\bar{Q})}$  where  $\Gamma_{\text{vars}(\bar{Q})}$  denotes the set of polymatroids over the variables  $\text{vars}(\bar{Q})$  [2]. Moreover for each finite input relation  $E \in \{R, S, T\}$ , we have  $\bar{h}(E) = 1$  since each one of these relations has four variables, i.e.  $|E| = 4$ . Therefore  $\bar{h}$  is edge dominated, i.e.  $\bar{h} \in \text{ED}(\bar{Q})$  where  $\text{ED}(\bar{Q})$  denotes the set of edge dominated functions  $h : 2^{\text{vars}(\bar{Q})} \rightarrow \mathbb{R}^+$ . Recall the definition of  $\text{subw}_\ell(Q)$  for an FAQ-AI query  $Q$  from [2] where  $\text{TD}_\ell(Q)$  denotes the set of relaxed tree decompositions of  $Q$ :

$$\text{subw}_\ell(Q) := \max_{h \in \text{ED}(Q) \cap \Gamma_{\text{vars}(Q)}} \min_{(\mathcal{T}, \chi) \in \text{TD}_\ell(Q)} \max_{t \in V(\mathcal{T})} h(\chi(t)). \quad (7)$$

Based on the above definition and by choosing  $\bar{h} \in \text{ED}(\bar{Q}) \cap \Gamma_{\text{vars}(\bar{Q})}$ , we have

$$\text{subw}_\ell(\bar{Q}) \geq \min_{(\mathcal{T}, \chi) \in \text{TD}_\ell(\bar{Q})} \max_{t \in V(\mathcal{T})} \bar{h}(\chi(t)).$$

However for each relaxed tree decomposition  $(\mathcal{T}, \chi) \in \text{TD}_\ell(\bar{Q})$ , we argued before that there must exist some bag  $t^* \in V(\mathcal{T})$  containing at least two of the input relations  $\{R, S, T\}$  hence at least 8 distinct variables, meaning that  $|\chi(t^*)| \geq 8$ . From (6), we have  $\bar{h}(\chi(t^*)) \geq 2$  which implies that  $\text{subw}_\ell(\bar{Q}) \geq 2$ . And since  $\text{subw}_\ell(Q) \leq \text{fhtw}_\ell(Q)$  for any query  $Q$  according to [2], we have

$$\text{subw}_\ell(\bar{Q}) = \text{fhtw}_\ell(\bar{Q}) = 2. \quad (8)$$

Finally according to Theorem 3.5 in [2], the time complexity in FAQ-AI involves an extra factor of  $(\log N)^{\max(k-1, 1)}$  where  $k$  is the number of inequalities that involve variables from two adjacent bags (i.e. that are not contained in a single bag) in an optimal relaxed tree decomposition. In query (5), when constructing any optimal relaxed tree decomposition involving two relations in one bag (say  $R$  and  $S$ ) and the third relation in another bag, there will be exactly 4 inequalities involving variables from both bags. Hence  $k = 4$  and the overall time complexity of FAQ-AI for  $Q_\Delta$  is  $O(N^2 \log^3 N)$ .